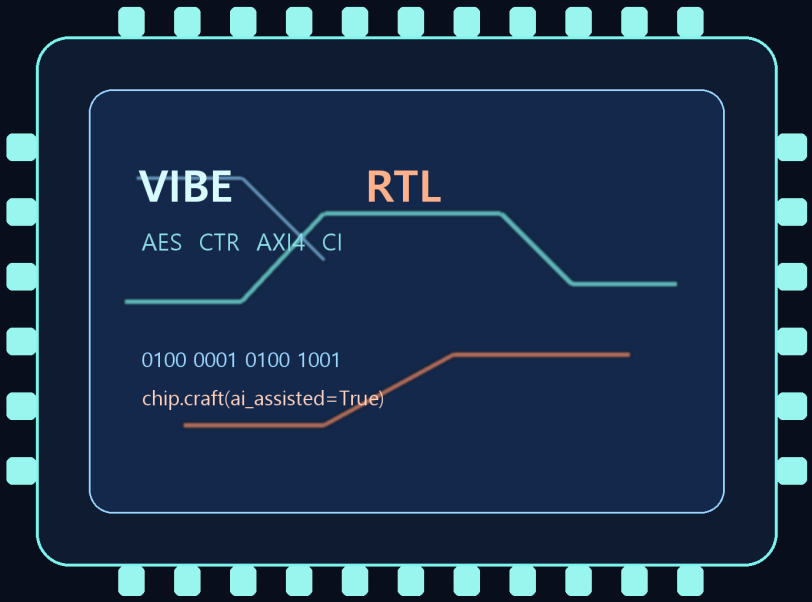
AI 한테칩설계시켜봤다

하드웨어엔지니어를위한, 생각보다현실적인바이브코딩실전입문

SSVD SoC Team

2026



AI한테 칩 설계 시켜봤다

하드웨어 엔지니어를 위한 생각보다 현실적인 바이브코딩 실전 입문

사양서부터 RTL, 테스트벤치, CI까지.
AI와 함께 3일 만에 AES IP를 만든 기록.

SSVD SoC Team

2026

차례

프롤로그—바이브코딩이하드웨어엔지니어에게오다 **8**

"이걸 AI 한테시켜도돼?" 8

바이브코딩이란무엇인가 9

이책의대상독자 10

개발환경 11

VSCode + Claude Code: 작업화면 11

이책을읽고나면 12

1 장—첫번째프롬프트, 거친아이디어를던지다 **13**

시작은메모한장이었다 13

원본프롬프트전문 13

이프롬프트가담고있는것 14

AI 와의첫대화—재구성 15

좋은첫프롬프트의조건 17

처음부터완벽한프롬프트는필요없다 18

프롬프트를던지는두가지방식 19

다음단계 20

2 장—AI 가사양서를쓰다 **20**

사양서가만들어지는방식이달라졌다 20

AI 가만들어온것 21

사양서의주요결정사항 22

사양서를다듬는과정—project-instructions.md 25

project-instructions.md 의구조 25

AI 협업에서이파일이중요한이유 26

완성된사양서의품질 27
사양서작성이끝난시점의판단 28
3 장—결정적전환, 설계까지맡겨보기로하다 28
처음에는사양서만목표였다 29
전환점이된세가지관찰 29
전환결정 31
바이브코딩전략을세우다 32
"바이브코딩"이"묻지마코딩"이아닌이유 34
한가지더: 사람이반드시검토해야하는것 34
4 장—프로젝트의골격을세우다 35
폴더구조가 AI 협업의품질을결정한다 35
확정된폴더구조 36
이구조의설계원칙 39
git 전략: 단순하게 40
룰설정이 AI 출력품질을높이는방법 41
구조를먼저만드는것의가치 42
5 장—RTL 을바이브코딩하다 43
하루만에 9 개모듈이나왔다 43
아키텍처결정: 어떻게분해할것인가 44
핵심모듈살펴보기 46
AI 에게 RTL 을요청할때의프롬프트패턴 54
첫커밋의코드통계 55
코드리뷰: AI 가스스로챙긴것들 56
한계: AI 가놓친것 57

6 장—레퍼런스소프트웨어를만들다 **58**
왜소프트웨어가먼저인가 58
Pure C 레퍼런스구현 59
IP 드라이버구조 61
테스트벡터생성 63
드라이버와사양서사이의교차검증 65
소프트웨어개발이 RTL 개발에미치는영향 66
7 장—검증환경을구축하다 **66**
테스트벤치도바이브코딩한다 67
시뮬레이션환경의구조 67
Fake Memory 설계—fake_mem.v 68
메모리초기화파일생성—gen_mem.c 70
테스트시나리오—tb_core.sv 71
NCVerilog 실행스크립트 75
Verilator 테스트벤치—오픈소스로의전환 75
테스트벤치가잡아내는것 77
8 장—버그를찾고고치다 **78**
버그는반드시온다 78
버그 1: sync_fifo 의이중구동 (Double Driver) 79
버그 2: SRAM 포트인터페이스불일치 81
버그 3: CRC 엔진초기화타이밍 83
AI 와함께버그를디버깅하는방법 85
버그수정이가르쳐주는것 86
commit dfed89d —이날추가된것들 87

9 장—오픈소스 CI 로자동화하다 **88**
왜 CI 가필요한가 88
Verilator vs NCVerilog 89
Verilator 테스트벤치구조 90
run.sh —빌드와실행을한번에 92
GitHub Actions 워크플로우 93
워크플로우설계의핵심결정들 96
CI 구축프롬프트 97
CI 가가져다주는것 98
10 장—다른 AI 를리뷰어로부르다 **98**
같은 AI 에게만물으면생기는문제 98
Codex 리뷰를위한준비 99
Codex 가발견한주요이슈들 101
Codex 리뷰의가치—정량적으로 105
AI 협업에서다중 AI 활용전략 105
리뷰어 AI 에게효과적으로요청하는법 106
에필로그—AI 동료와일한다는것 **107**
3 일의기록을돌아보며 107
바이브코딩이잘맞는영역과그렇지않은영역 107
엔지니어의역할이바뀐다 108
이방식으로개발을시작하려는분께 109
마지막으로 110
부록: 이책에서사용한프롬프트패턴요약 111

프롤로그—바이브코딩이하드웨어엔지니어에게오다

“이걸 AI 한테시켜도돼?”

RTL 설계를 15 년넘게해온엔지니어도처음엔반신반의한다.
솔직히말하면반신반의정도가아니라, 속으로는이미절반쯤“이건안되겠는데”라고결론내린상태에가깝다.

> “파이썬스크립트나간단한유틸정도는 AI 한테맡길수있겠지.
> 그런데 AXI4 프로토콜을따르는 FSM? 파이프라인 AES 코어?
> SRAM 인터페이스가달린 FIFO? 그건좀다른얘기아닌가?”

이책은그의심에대한실전답변이다.
“AI 가진짜여기까지할수있나?”라는질문을추상적인전망이아니라, 실제프로젝트기록으로붙잡아보려는시도이기도하다.

2026 년 4 월, 저자는 AES 복호화하드웨어 IP 를만들어야했다.
처음에는 **구현사양서**문서만 AI 에게부탁했다.
그런데 AI 가만들어온사양서를보고나서생각이바뀌었다.

“이정도퀄리티라면…설계자체도맡겨볼수있지않을까?”

그판단이맞았는지틀렸는지, 그리고어디까지는놀랍고어디부터는사람

이꼭붙어있어야하는지.
이책은그 3 일의기록이다. 약간의감탄과, 적당한식은땀, 그리고생각보다많은커밋로그가함께들어있다.

바이브코딩이란무엇인가

"바이브코딩 (Vibe Coding)"은 2025 년 AI 연구자 Andrej Karpathy 가처음사용한표현이다. 정확한정의보다는느낌이중요하다.

> 자신이원하는것을자연어로말하고, AI 가코드를만들어주는것.
> 세부구현은 AI 에게맡기고, 사람은방향과판단에집중하는개발방식.

소프트웨어세계에서는이미상용레벨의프론트엔드, API 서버, 스크립트를 AI 가만들어내고있다. 하드웨어는어떨까?

RTL 은다르다. 단순히"동작하는코드"로끝나지않는다.

- 클록도메인과타이밍을알아야한다
- AXI 프로토콜핸드셰이크를정확히구현해야한다
- Divider 나 wide multiplier 는합성시타이밍위반을낸다
- SRAM 인터페이스와 FIFO 제어로직은미묘한코너케이스가있다
- 테스트벤치도, 합성스크립트도, 드라이버코드도필요하다

그러나바로그렇기때문에, 이모든지식을이미학습한 AI 와협업하면상황이달라진다.

사람은방향과기준을잡고, AI 는초안과반복작업을밀어준다. 이조합이맞아떨어지면, 혼자서는엄두가안나던규모의작업도꽤현실적인일정으로끌어올수있다.

조금과장해서말하면, 예전에는"이걸하려면팀이필요하다"라고생각했던일이이제는"이걸하려면팀같은 AI 와꽤끈질긴엔지니어가필요하다" 정도로바뀐다.
둘은비슷해보이지만, 실제체감속도는꽤다르다.

이책의대상독자

이책을가장잘활용할수있는독자는다음과같다.

- RTL 설계경험이있는 SoC·IP 엔지니어
- AXI, AES, DMA, FIFO 같은개념이낯설지않은사람
- AI 를써보고싶지만"하드웨어에도통할까?"라고의심하는사람
- "바이브코딩"이라는단어는들었지만실제로어떻게하는지모르는사람

반대로, Verilog 나 AXI 를전혀모른다면이책의 RTL 챕터는다소어려울수있다.
그러나바이브코딩의흐름과 AI 협업방식은충분히따라올수있을것이다.

개발환경

이책에서사용한도구는간단하다.

도구	역할
VSCode	에디터
Claude Code Extension	AI 코딩어시스턴트 (Anthropic Claude Sonnet)
Git	버전관리
Verilator	오픈소스시뮬레이터 (CI 용)
GitHub Actions	CI 파이프라인

NCVerilog 나 Synopsys DC 같은 EDA 툴은원래환경에있었다.
하지만이책에서다루는바이브코딩흐름자체는어떤 EDA 환경에서도동일하게적용된다.

VSCode + Claude Code: 작업화면

> [IMG] **[그림 0-1]** VSCode 에 Claude Code Extension 이설치된작업화면
> 왼쪽: 파일탐색기 (프로젝트폴더구조), 오른쪽: Claude Code 사이드패널 (대화창)

Claude Code 는 VSCode 의사이드패널에서대화형식으로작동한다.
일반채팅창과다른점이있다면, **프로젝트폴더전체가컨텍스트**로연결되

어있다는것이다.

- AI 가현재열려있는파일을직접읽는다
- 코드를수정할때는에디터에바로반영된다
- 터미널명령도 AI 가제안하고, 승인하면실행된다

  [IMG] **[그림 0-2]** Claude Code 의파일읽기/쓰기권한승인팝업
  "Allow"/ "Deny"버튼이있으며, 파일변경시 diff 미리보기가함께표시된다

이책전반에걸쳐이인터페이스를통해 AI 와주고받은핵심대화를재구성해보여준다.

이책을읽고나면

바이브코딩은마법이아니다. AI 가모든것을알아서해주지않는다.
오히려가끔은너무자신있게틀리기때문에, 사람입장에서는"야, 그건좀아니다"라고말할준비도되어있어야한다.

하지만 **어떻게질문하고, 어떻게방향을잡고, 어떻게검증하는지**를알면, 혼자서는엄두도못냈을작업을현실적인시간안에완성할수있다.
이책은그감각을최대한구체적으로보여주려한다. "AI 가해줬다"가아니라, "어떻게같이굴렸는가"에더가깝게.

이책이그첫걸음을보여준다.

다음장: 1 장—첫번째프롬프트, 거친아이디어를던지다

1 장—첫번째프롬프트, 거친아이디어를던지다

시작은메모한장이었다

모든설계는누군가의머릿속에서시작된다. 처음엔완전하지않아도된다.
대개는화이트보드낙서이거나, 메신저에급히적어둔문장이거나, "내일아침엔이해하겠지"라는희망이담긴메모다.

이프로젝트도마찬가지였다. 시작은정리되지않은요구사항, 메모에가까운텍스트였다.
그러니까말하자면, 엔지니어링의출발점치고는꽤전형적이었다. 멋진아키텍처다이어그램대신, 약간급한마음이먼저있었다.

파일이름: `prompt/original-prompt.md`

원본프롬프트전문

이번에 너가 할일은 사양체크리스트 문서 작성은 아니고 칩업체의 IP가 설계할 AES decryption hardware 엔진의 개발사양서 문서야.

이 IP는 memory to memory 방식으로 동작하고 버스 마스터 및 레지스터

읽기 쓰기를 위한 슬레이브 인터페이스를 갖고 있어. 인터럽트 출력도
버스 인터페이스는 IP개발자가 결정하면 돼.

SW가 메모리의 명령 버퍼(circular buffer)에 적어주는 유효한 Descri
읽어서 명령을 처리하고 처리완료하면 디스크립터의 상태 필드에 완료
디스크립터의 valid 플래그도 클리어해. 해당 디스크립터에 인터럽트
나타내는 필드가 1 세팅되어 있다면 인터럽트를 띄우고 SW가 핸들링완
레지스터를 1로 써줄때까지 대기해.

...

AES는 128bit CTR체인모드로 decrypt해야해. AES 헤더(IV와 AES CTR 표
decrypt에 필요한 데이터)가 먼저오고 encrypted data와 패딩이 이어

throughput 성능 요구도 사양서에 기술되어야해.

이 같은 내용으로 IP 목적 동작설명을 인트로에 서술하고 SW와 인터페
위한 레지스터, 명령/데이터/버퍼 레이아웃과 종류, 인터럽트 핸들링
IP 구현 요청 사양서를 word와 markdown으로 작성해줘.

이프롬프트가담고있는것

프롬프트를분해해보면, 다음정보들이섞여있다.

담긴정보	구체성
IP 종류 (AES decrypt, memory-to-memory)	명확함
동작방식 (circular buffer, descriptor)	명확함
인터럽트동작 (PAUSE/RESUME 흐름)	비교적명확함
버스인터페이스	**"개발자가결정"**—미정
성능목표 (throughput)	**수치없음**
CRC, 패딩, AES 헤더상세	**불완전**

즉, **개념은있지만디테일이빠진**상태다.
보통이런문서는회의한번, 메일두번, 그리고"이건누가결정하죠?"라는질문세번정도를더거쳐야비로소사양서처럼보이기시작한다.

일반적인경우라면이상태에서설계를시작하면나중에많은것을되짚어야한다.
AI 와의협업에서흥미로운지점은여기다. AI 는빈칸을억지로메우거나, 최소한"여긴아직비어있습니다"라고표시라도해준다.
사람이해야할일은그제안을평가하고, 틀린자신감을제때잡아주는것이다.

AI 와의첫대화—재구성

[IMG] **[그림 1-1]** VSCode 사이드패널에 original-prompt.md 내용을붙여넣어 Claude 에게전달하는화면

사용자 →

[original-prompt.md 전문 붙여넣기]

일단 이 내용을 바탕으로 IP 구현 사양서를 작성해줘.
빠진 부분은 IP 개발자 관점에서 합리적인 선택을 해서 채워줘.

Claude →

사양서 작성 전에 몇 가지 결정이 필요한 사항을 확인드립니다.

1. 버스 인터페이스: AXI4 (64-bit Manager + AXI4-Lite Subordinate)로 제안합니다.
   - 200 Mbps 목표에는 64-bit AXI4가 적합합니다.
   - 레지스터 인터페이스는 AXI4-Lite로 분리하는 것이 표준적입니다.

2. 처리량 목표: 200 Mbps를 기준으로 잡겠습니다.
   (나중에 변경 시 모든 문서에 반영해야 하므로 THROUGHPUT_TARGET 매

3. AES 키 저장: Descriptor가 아닌 IP 레지스터에 저장 (보안 및 재사

4. CRC: CRC-32/IEEE 802.3와 CRC-32C 둘 다 지원, 레지스터로 선택

5. Descriptor 크기: 32바이트 (32-bit word aligned)

이 방향으로 진행해도 될까요?

사용자 →

맞아. 그 방향으로 진행해줘.

[IMG] **[그림 1-2]** Claude Code 가 spec/ 폴더에 AES-Decrypt-IP-Specification.md 파일을생성하는과정
파일쓰기권한승인팝업이표시되고, 승인후에디터에파일이열린다

좋은첫프롬프트의조건

이프롬프트가효과적이었던이유를정리하면다음과같다.

1. 목적이명확했다

"사양체크리스트가아니라구현사양서"라고구체적으로말했다.
AI 는문서의성격을즉시파악했다.

2. 동작흐름을글로설명했다

비록완전하지않았지만, descriptor 의 valid 체크 → 처리 → 인터럽트 → PAUSE → RESUME 흐름을서술했다. AI 는이흐름을상태머신으로변환했다.

3. 빈칸을명시적으로허용했다

"버스인터페이스는 IP 개발자가결정하면돼"라고했다.
이표현이 AI 에게결정권을명시적으로넘겨준것이다. 애매하게두지않았다.

4. 출력형식을요청했다

"markdown 으로작성해줘"라고구체적으로요청했다.
Word 와 Markdown 을동시에요청했지만, 실제로는 Markdown 만생성했고이후수정하지않았다.

처음부터완벽한프롬프트는필요없다

하드웨어엔지니어는완벽한사양이나올때까지설계를시작하지않는다는훈련을받아왔다.
그태도는대부분의경우옳다. 실제칩은"대충"이라는단어를별로좋아하지않는다.

바이브코딩에서는이습관을조금내려놓아야한다.
정확히는완벽주의를버리는것이아니라, 초안작성의첫번째삽을 AI 에게맡기는쪽에가깝다.

> **"70% 의아이디어를던지고, 나머지 30% 는 AI 와함께채워가라."**

AI 는빠진부분을물어보거나, 합리적인가정을세워진행한다.

가끔은너무당당하게가정하기도하지만, 그조차도빈화면을바라보는것보다는생산적이다.

첫프롬프트의역할은 **방향을정하는것**이지, 모든것을정의하는것이아니다.
첫공을너무완벽하게던지려하기보다, 스트라이크존안으로만넣자는마음가짐이오히려잘통한다.

프롬프트를던지는두가지방식

실전에서는두가지스타일이존재한다.

스타일 A: 개방형

```
AES decrypt IP를 만들어야 해. Memory-to-memory 방식이고
Descriptor 기반으로 동작해. 사양서를 작성해줘.
```

→ AI 가많은것을결정한다. 빠르지만의도와다를수있다.

스타일 B: 반구조형 (이책에서사용)

```
[핵심 동작 설명 + 결정된 사항 나열 + 열린 부분 명시]
빠진 부분은 합리적인 선택으로 채워줘.
```

→ AI 가채울여지를주면서도방향은잡아둔다. 하드웨어처럼제약이많은도메인에적합하다.

이책에서는일관되게 **스타일 B** 를사용했다.

다음단계

AI 가사양서의초안을만들어왔다. 하지만그것은시작일뿐이었다.
진짜작업은그사양서를읽고, 확인하고, 다듬는과정에서시작된다.

다음장: 2 장—AI 가사양서를쓰다

2 장—AI 가사양서를쓰다

사양서가만들어지는방식이달라졌다

전통적인 IP 개발에서사양서는가장시간이걸리는문서다.
요구사항을수집하고, 인터페이스를정의하고, 레지스터맵을그리고, 동작다이어그램을그리고, 용어를통일하고, 다시처음으로돌아가빠진항목을메우고...
경험많은엔지니어도며칠에서일주일이걸린다. 그리고그며칠은대체로생각보다빨리지나간다.

이프로젝트에서는달랐다. AI 가초안을만드는데걸린시간은 **수분**이었다.
문서의품질과는별개로, 이속도자체가주는심리적이점이크다. "아직아무것도없다"가아니라"이제부터고치면된다"가되기때문이다.

물론초안이곧최종본은아니다. 그러나 **빈화면앞에서시작하지않아도**

된다는것은완전히다른경험이다.

AI 가만들어온것

Claude 가생성한사양서 (spec/AES-Decrypt-IP-Specification.md)
의구조를보면, 전문적인 IP 사양서의형식을따르고있다.

```
1. Introduction        ← 목적, 범위, 대상 독자, 참조 문서
2. Terminology         ← 용어 정의 (Master→Manager, Slave→Subordina
3. System Overview     ← 블록 다이어그램, 주요 기능
4. Interface Signals   ← AXI4 Manager / AXI4-
Lite Subordinate / IRQ 전체 핀
5. Register Map        ← 17개 레지스터, 비트 단위 정의
6. Descriptor Layout   ← 32바이트 레이아웃, 입력 버퍼 구조
7. Operational State Machine ← STOP / ACTIVE / PAUSE 전환 규칙
8. AES Processing      ← CTR 모드 동작, 키 스케줄
9. CRC Processing      ← 두 알고리즘 파라미터, 에러 처리
10. Interrupt Handling ← 타이밍 다이어그램 포함
11. Performance Requirements ← 200 Mbps 근거와 설계 제약
12. Bus Interface Specification ← AXI 세부 규칙
13. Electrical and Physical Constraints ← 합성 가능성 요구사항
14. Revision History
```

14 개섹션, 900 라인분량의문서가단번에나왔다.
문서를다읽기전에먼저"잠깐, 이걸정말방금만든거야?"라는반응이나

오는종류의결과물이었다.

사양서의주요결정사항

AI 가채운핵심결정들을확인하고검토했다. 각항목은 **확인 (□)** 또는 **수정 (□)** 으로처리했다.
중요한건감탄이아니라판정이다. "오, 잘썼네"에서끝내지않고, 정말이 선택이맞는지하나씩도장을찍어야한다.

버스인터페이스 □

```
AXI4 Manager: 64-bit 데이터폭, 32-bit 주소
AXI4-Lite Subordinate: 32-bit 데이터폭, 8-bit 주소 (256바이트 레
```

64-bit 선택이유는사양서에명시되어있다.

> "At 200 MHz with 64-bit AXI bus: peak AXI bandwidth = 1600 MB/s = 12.8 Gbps.
> The 200 Mbps throughput requirement represents a light load on the bus."

200 Mbps 목표대비버스대역폭은충분하고, 64-bit 는 ASIC SoC 에서 흔한선택이다.

레지스터맵 □ (일부확인필요)

AI 가설계한레지스터맵의일부:

Offset	Name	Access	설명
0x00	CTRL	W (self-clear)	START / RESUME / IMMEDIAT
0x04	STATUS	Mixed	STATE[1:0] RO, BUS_ERROR W
0x10	CMD_BUF_ADDR	R/W	Descriptor 링버퍼베이스주소
0x20~0x2C	AES_KEY_[3:0]	WO	AES-128 키 (읽기시 0 반환)
0x34	AXI_OUTSTAND	R/W	Outstanding 트랜잭션수제한

AES 키레지스터가 **WO(Write Only)** 로설계된것은보안관점에서당연한선택이다.
CTRL 레지스터의비트들이 **self-clearing pulse** 로설계된것도하드웨어설계관례에맞다.

Descriptor 레이아웃 □

```
Byte Offset  Size     Field
──────────────────────────────────────────
0x00       4 B    Header Word (valid, interrupt, last, state)
0x04          4 B       Input Buffer Address
0x08          4 B       Output Buffer Address
0x0C       4 B     IN_DATA_SIZE[23:0] + IN_PAD_SIZE[7:0]
0x10       4 B    OUT_DATA_SIZE[23:0] + OUT_PAD_SIZE[7:0]
0x14         12 B       Reserved
──────────────────────────────────────────
Total:        32 B
```

Header Word 를 4 개바이트로분할해서 AXI Write Strobe 로개별업데이트가능하게한것은 AI 의결정이었다. IP 가 `valid=0` 으로클리어할때 `state` 필드와충돌하지않도록하기위해서다. 실제 ASIC 설계에서도자주쓰이는패턴이다.

입력버퍼레이아웃—중요한결정 □

```
Input Buffer (at IN_ADDR)
──────────────────────────────────────────────────────────
+0x00  16 B  AES Header (Nonce 96-bit + Initial Counter 32-bit)
+0x10    N B   Encrypted Payload (ciphertext)
+0x10+N  M B  Input Padding (버스 정렬용, CRC 계산 대상 아님)
+0x10+N+M 4 B  CRC-32 Value
──────────────────────────────────────────────────────────
```

CRC 를 **암호화된데이터 (ciphertext)** 에대해계산하는설계는중요한결정이다.
복호화후의 plaintext 에 CRC 를계산하면 AES 처리완료후에야검증이가능하다.
Ciphertext 에 CRC 를계산하면 **AES 복호화와 CRC 계산을병렬로실행**할수있다.
AI 가이트레이드오프를이해하고올바른선택을했다.

사양서를다듬는과정—project-instructions.md

초안사양서를검토한후, 저자는 AI 와의작업규약을정리한별도파일을만들었다.
`prompt/project-instructions.md` 가그것이다. 쉽게말해, AI 와의협업에서자꾸되풀이해말해야하는것들을한장짜리헌법으로묶어둔셈이다.

이파일은두가지역할을한다.

1. **확정된결정사항의기록** —이후변경시여기를먼저업데이트
2. **AI 에게주는지속적인지시** —매대화마다이파일을참조하도록

[IMG] **[그림 2-1]** project-instructions.md 를 Claude Code 에컨텍스트로추가하는화면
파일을드래그하거나 `@` 파일명으로참조하면 AI 가해당파일내용을읽는다

---

project-instructions.md 의구조

```
## 프로젝트 폴더 구조
| 폴더/파일 | 내용 |
|---|---|
| prompt/   | 이 문서. 전체 지시 및 확정 결정사항 저장 |
| spec/     | 사양 문서 결과물 |
| design/   | Verilog — DUT, Testbench, Task, Test data |
```

```
| host_software/ | Host C code |

## 기타 룰
- 어느 하나에 수정이 있을 경우 프로젝트 내 모든 코드, 문서가 일관성을
- 문서에서 회사명은 SSVD, 팀명은 SoC team
- Manager(=Master), Subordinate(=Slave) 표기
- 모든 작성물은 영어로 작성

## 확정 사양
- AXI4 64-bit Manager
- 목표 처리량: 200 Mbps
- CRC-32 dual (IEEE 802.3 / CRC-32C, 레지스터 선택)
- AES 키: IP 레지스터에 저장
- Descriptor 최대 1024 개
```

AI 협업에서이파일이중요한이유

Claude Code (그리고대부분의 AI 코딩어시스턴트) 는 **대화세션이끊기면이전맥락을잃는다**.
새세션을시작할때마다"우리가어디까지했는지"를다시알려줘야한다.

`project-instructions.md` 는이문제에대한해답이다.
세션이바뀔때마다같은설명을처음부터다시하는일은사람끼리도지치는데, AI 상대로하면더빨리지친다.

```
# 새 세션 시작 시 첫 메시지 패턴
"prompt/project-instructions.md를 먼저 읽어줘.
이 프로젝트의 확정된 사양과 룰이 담겨 있어.
이후 작업은 이 파일을 기준으로 진행해."
```

AI 는이파일을읽고, 이전결정사항을모두파악한뒤작업을시작한다.
이렇게하면매번컨텍스트를다시설명하는시간을절약할수있다.

완성된사양서의품질

완성된사양서의수준을가늠하는지표몇가지:

아키텍처일관성

레지스터맵, Descriptor 레이아웃, 상태머신이서로충돌하지않는다.
예를들어, `interrupt=1` 인 Descriptor 가완료되면 PAUSE 상태로진입하고,
`IRQ_STATUS.DESCRIPTOR_DONE` 비트가 set 되며, `CTRL.RESUME=1` 을받아야 ACTIVE 로복귀한다.
이흐름이레지스터설명, 상태머신표, 인터럽트핸들링섹션에서모두일관되게서술되어있다.

ASIC 고려사항포함

```
## 13. Electrical and Physical Constraints
- Dividers and wide multipliers are prohibited.
- No use of initial blocks except in testbench files.
- No combinational loops.
```

ASIC 합성환경에서문제가되는제약들이명시되어있다.

성능분석포함

```
At 200 MHz with 64-bit AXI bus:
peak AXI bandwidth = 200 MHz × 8 B = 1600 MB/s = 12.8 Gbps.
The 200 Mbps throughput requirement represents a light load on
```

200 Mbps 가왜달성가능한지수치로설명하고있다.

사양서작성이끝난시점의판단

사양서를검토하면서한가지생각이강하게들었다.
조금위험하고, 그래서더매력적인생각이었다.

> "이정도사양서를만들수있는 AI 라면, RTL 코드도충분히만들수있지않을까?"

이것이다음장에서다루는 **결정적전환**으로이어진다.

다음장: 3 장—결정적전환, 설계까지맡겨보기로하다

3 장—결정적전환, 설계까지맡겨보기로하다

처음에는사양서만목표였다

이프로젝트를시작할때의목표는단순했다.

> "IP 구현사양서를빠르게만들자. 실제설계는나중에, 사람이 한다."

하드웨어엔지니어에게이것은자연스러운생각이다.
RTL 코드는정밀해야하고, 프로토콜위반하나가칩전체를망가뜨릴수있다.
"AI 가그걸제대로할수있을까?"라는의심은당연했다.
정확히는, 의심하지않는쪽이오히려이상하다.

그런데사양서가나오고나서생각이바뀌었다.

전환점이된세가지관찰

관찰 1: 레지스터맵이자연스럽다

AI 가설계한레지스터맵에는숙련된엔지니어의판단이담겨있었다.

- `CTRL` 레지스터의비트를 self-clearing pulse 로설계한것
- `AES_KEY` 레지스터를 Write-Only 로만들어보안을고려한것
- `CMD_HEAD_PTR` 는 Read-Only, `CMD_TAIL_PTR` 는 SW 가쓰는구조로나눈것
- W1C(Write-1-to-Clear) 패턴을정확하게적용한것

이것들은하드웨어레지스터설계의표준패턴들이다.

한두개쯤맞히는건우연일수있지만, 이런패턴이연달아자연스럽게나오면이야기가달라진다.
AI 가이패턴들을정확하게선택했다는것은, 적어도이문제를"그럴듯하게흉내"내는수준이상이라는뜻이었다.

관찰 2: 상태머신이엣지케이스를다룬다

일반적인 AI 출력물에서자주빠지는것이엣지케이스처리다.
그런데이사양서에는다음이명시되어있었다.

```
If last=1 and interrupt=1:
  → 엔진이 먼저 PAUSE 상태에 진입
  → SW가 CTRL.RESUME=1을 쓴 후
  → ACTIVE가 아니라 STOP으로 전환 (다음 Descriptor를 처리하지 않음
```

last 와 interrupt 가동시에 1 인경우를정확하게처리하고있다.
이케이스는설계자가직접생각해도빠뜨리기쉬운부분이다.

관찰 3: 성능설계가구체적이다

단순히"200 Mbps 를만족해야한다"고쓴것이아니라,
그것을달성하기위한 **아키텍처제약**까지명시했다.

```
## 11.2 Design Requirements to Achieve Throughput Target

1. AES core must not starve: Input FIFO must be large enough to
   AXI read latency. Begin prefetching as soon as IN_ADDR and I
   are decoded, without waiting for full descriptor.
```

```
2. No unnecessary bus congestion: Do not issue more read transa
   needed to fill the internal input FIFO.

3. Descriptor overhead minimized: Fixed-size portion fetched in
   AXI burst to minimize read latency.
```

이것은단순한요구사항이아니라 **설계방향**이다.
숫자만있는사양서가아니라, 왜그숫자를만족할수있는지까지설명한문서라는점이중요했다.
이내용이사양서에있다면, RTL 코드에도그대로반영할수있다.

전환결정

이세가지를확인한뒤, 질문이바뀌었다.

> "AI 가이사양서를이해하고쓸수있다면,
> 이사양서를입력으로 RTL 을만들수도있지않을까?"

그리고한가지더, 이미사양서가있다.
RTL 을만들기위한 **레퍼런스문서**가완성되어있는것이다.
즉"뭘만들지"를다시설명하는대신, "이문서를근거로구현해"라고말할수있다. 이차이는꽤크다.
프롬프트가감정노동에서지시문으로바뀌기때문이다.

바이브코딩전략을세우다

무작정"Verilog 로만들어줘"라고하지않았다.

RTL 바이브코딩을시작하기전에 **전략**을먼저잡았다.

전략 1: 모듈을분해하고순서를정한다

AI 에게전체를한번에요청하지않는다.

복잡한 IP 는모듈로분해해서하나씩만들고, 조립한다.

```
작업 순서:
1. aes_decrypt_defs.vh  ← 파라미터, 상수 정의 (가장 먼저)
2. aes_decrypt_regfile.v ← 레지스터 파일 (가장 독립적)
3. aes128_key_sched.v    ← AES 키 스케줄 (암호화 코어)
4. aes128_enc_pipe.v     ← AES 파이프라인 코어
5. aes128_ctr_top.v      ← CTR 모드 래퍼
6. crc32_engine.v        ← CRC 엔진
7. sync_fifo.v           ← 범용 FIFO
8. aes_decrypt_desc_fetch.v ← Descriptor 읽기
9. aes_decrypt_input_ctrl.v ← 입력 버퍼 컨트롤러
10. aes_decrypt_output_ctrl.v ← 출력 버퍼 컨트롤러
11. aes_decrypt_writeback.v ← Descriptor 상태 쓰기
12. aes_decrypt_axi_mgr.v ← AXI Manager (모든 채널 중재)
13. aes_decrypt_ctrl.v   ← 최상위 FSM (가장 마지막)
14. aes_decrypt_engine.v ← 최상위 래퍼 (배선)
```

의존성이낮은모듈부터시작해서, 복잡한것은나중에만드는방식이다.

전략 2: 사양서를항상참조한다

```
# 각 모듈 작성 시 프롬프트 패턴
"spec/AES-Decrypt-IP-Specification.md의 [섹션 번호]를 참고해서
[모듈명].v를 작성해줘.
합성 가능해야 하고, ASIC 환경에서 문제없어야 해."
```

전략 3: 모듈마다헤더주석형식을통일한다

```
// ============================================================
// File        : 파일명.v
// Project     : AES Decryption Engine IP
// Company     : SSVD
// Description : 모듈 설명
// ============================================================
```

이형식을처음에한번정해두면, AI 가이후모든파일에서동일하게유지한다.

전략 4: project-instructions.md 에설계룰을추가한다

```
## IP 설계 요구사항 (Verilog)
1. 사양서대로 하드웨어를 Verilog 로 구현
2. 고성능 파이프라인 구조:
   - Descriptor 고정 크기는 한 번에 읽어 해석
   - IN_ADDR, IN_DATA_SIZE 확인 즉시 미리 read 시작
3. 합성 가능, ASIC 환경에서 문제없어야 함
4. Divider, Wide-bit multiplier 사용 금지
5. 생산 가능한 수준의 완성도
```

```
6. 옵션으로 on/off 가능한 (`define) SVA Assertion 및 Coverage 추가
```

이룰이 `project-instructions.md` 에들어가면,
이후모든 Verilog 작성요청에서 AI 가자동으로이기준을따른다.

"바이브코딩"이"묻지마코딩"이아닌이유

여기서한가지오해를짚고싶다.

바이브코딩은 AI 에게무조건맡기는것이아니다.
오히려 **더많은판단**이엔지니어에게요구된다.
손이덜바빠지는대신, 머리는더자주깨어있어야한다.

- 모듈을어떻게분해할것인가?
- 어떤인터페이스로연결할것인가?
- AI 가만든코드가사양과일치하는지검토할수있는가?
- 버그가생겼을때어디를보아야하는가?

이판단들은모두도메인지식에서나온다.
바이브코딩은도메인지식이필요없는방식이아니라,
도메인지식있는엔지니어의생산성을극대화하는방식이다.

한가지더: 사람이반드시검토해야하는것

설계를 AI 에게맡기더라도, 결과를사람이검토해야하는부분이있다.

1. **AXI 프로토콜준수여부** —handshake 타이밍, outstanding count 처리
2. **리셋후초기상태** —모든레지스터가사양서의 reset value 와일치하는지
3. **엣지케이스** —FIFO 가가득찬상태에서쓰기요청이오는경우등
4. **합성경고** —AI 가만든코드가래치 (latch) 를만들지않는지

이것들은코드리뷰체크리스트로만들수있다.
그리고체크리스트자체도 AI 에게만들어달라고할수있다.

다음장: 4 장—프로젝트의골격을세우다

4 장—프로젝트의골격을세우다

폴더구조가 AI 협업의품질을결정한다

바이브코딩에서폴더구조는단순한파일정리이상의의미를가진다.
AI 는파일을읽을때 **경로와위치**로맥락을파악한다.
사람이"아, 이건테스트용이구나"하고눈치로이해하는것을, AI 는폴더이름과파일배치로배운다.

`design/rtl/` 폴더에있는 `.v` 파일은 RTL 소스다.
`design/tb/` 폴더에있는파일은시뮬레이션환경이다.
`host_software/` 폴더에있는 `.c` 파일은 C 드라이버코드다.

폴더구조가명확하면 AI 에게"tb_top.v 를수정해줘"라고했을때
AI 가스스로 `design/tb/tb_top.v` 를찾아읽고작업한다.
구조가모호하면 AI 는추측하거나물어봐야한다.

확정된폴더구조

```
Decrypt-AES/
│
├── prompt/
│   ├── project-instructions.md  ← 전체 규약 + 확정 결정사항 (AI 컨
│   └── original-prompt.md      ← 최초 요청 원문 (기록용)
│
├── spec/
│   └── AES-Decrypt-IP-Specification.md  ← IP 구현 사양서
│
├── design/
│    ├── rtl/
│    │    ├── inc/
│   │   │    └── aes_decrypt_defs.vh     ← 전역 파라미터, define
│    │    ├── crypto/
│   │   │    ├── aes128_key_sched.v     ← AES 키 스케줄 (조합 논리)
│   │   │    ├── aes128_enc_pipe.v      ← AES 10라운드 파이프라인
│    │   │    └── aes128_ctr_top.v         ← CTR 모드 래퍼
│    │    ├── util/
```

```
│   │   │   ├── sync_fifo.v            ← 범용 동기 FIFO
│   │   │   ├── crc32_engine.v          ← CRC-32 엔진
│   │   │   ├── sram_2p.v                    ← 2-
port SRAM 기본 모델
│   │   │   ├── sram_2p_32x64.v       ← 32x64 SRAM 인스턴스
│   │   │   └── sram_2p_32x72.v       ← 32x72 SRAM 인스턴스
│   │   ├── aes_decrypt_engine.v       ← 최상위 (DUT top)
│   │   ├── aes_decrypt_regfile.v           ← AXI4-
Lite 레지스터 파일
│   │   ├── aes_decrypt_ctrl.v           ← 최상위 FSM
│   │   ├── aes_decrypt_desc_fetch.v    ← Descriptor 읽기
│   │   ├── aes_decrypt_input_ctrl.v    ← 입력 버퍼 컨트롤러
│   │   ├── aes_decrypt_output_ctrl.v   ← 출력 버퍼 컨트롤러
│   │   ├── aes_decrypt_writeback.v     ← Descriptor 상태 쓰기
│   │   ├── aes_decrypt_axi_mgr.v      ← AXI4 Manager (2R+2W)
│   │   └── aes_decrypt_mem_top.v      ← SRAM 집합체 (MBIST 경계)
│   │
│   ├── tb/
│   │   ├── tb_top.v        ← 시뮬레이션 최상위
│   │   ├── tb_core.sv     ← 테스트 시나리오 (task 모음)
│   │   ├── tb_defines.vh   ← 테스트벤치 파라미터
│   │   ├── fake_mem.v     ← AXI4 Subordinate 메모리 모델
│   │   ├── gen_mem.c      ← 메모리 초기화 파일 생성기 (C)
│   │   ├── mem_init.hex    ← readmemh 초기화 파일
│   │   └── run.sh          ← NCVerilog 실행 스크립트
```

```
│   │
│   ├── tb_verilator/
│   │   ├── tb_top_verilator.sv  ← Verilator용 TB 최상위
│   │   ├── tb_dpi.cpp           ← C++ 시뮬레이션 드라이버
│   │   ├── run.sh               ← Verilator 빌드/실행 스크립트
│   │   └── README.md
│   │
│   └── syn/
│       ├── run_dc.tcl       ← Synopsys DC 합성 스크립트
│       └── constraints.sdc  ← 타이밍 제약 (SDC)
│
├── host_software/
│       ├── aes128_ctr.c/.h          ← Pure C AES-
128 CTR (레퍼런스)
│   ├── crc32.c/.h           ← Pure C CRC-32 (레퍼런스)
│   ├── aes_decrypt_ip.c/.h ← IP 레지스터 드라이버
│   ├── test_vectors.h     ← 테스트 벡터 (암호화 데이터 + 키)
│   ├── tc_params.h          ← 테스트 케이스 파라미터
│   ├── ip_test.c        ← IP 사용 테스트 (드라이버 기반)
│   └── sw_test.c            ← 순수 SW 구현 테스트
│
├── doc/
│   ├── AES-Decrypt-IP-Architecture-Description.md
│   └── compiled_memory_list.txt
│
```

```
└── README.md                    ← 프로젝트 개요, 빠른 시작
```

이구조의설계원칙

원칙 1: 용도별분리

`design/rtl/` 에는합성에들어가는소스만있다.
`design/tb/` 에는시뮬레이션전용코드만있다.
`design/syn/` 에는합성스크립트만있다.

이분리는 AI 에게도명확한신호가된다.
"tb_top.v 수정"이라고했을때 AI 가 RTL 소스를건드리지않는다.
디렉터리구조가어수선하면 AI 도어수선하게일한다. 사람과크게다르지않다.

원칙 2: 계층구조

`design/rtl/` 안에 `crypto/`, `util/`, `inc/` 하위폴더를두었다.
이는모듈의역할을계층으로표현한것이다.

- `inc/` —전역정의 (모든모듈이포함)
- `util/` —재사용가능한기본블록 (FIFO, CRC, SRAM)
- `crypto/` —암호화전용코어
- 루트—프로젝트특화모듈

원칙 3: prompt/ 폴더의역할

이폴더는일반적인 IP 프로젝트에는없는폴더다.
바이브코딩프로젝트에서이폴더는 **AI 와의계약서역할**을한다.
물론법적효력은없지만, 체감상으로는꽤강력하다.

`project-instructions.md` 파일에는다음이담긴다.

- 확정된설계결정사항
- 문서작성규칙 (회사명, 용어표기방식)
- 코드작성규칙 (Divider 금지, SVA 추가등)
- 폴더구조와각폴더의역할

새대화를시작할때이파일하나만참조하면 AI 가즉시프로젝트맥락을이해한다.

---

git 전략: 단순하게

복잡한브랜치전략은쓰지않았다.
바이브코딩에서는생성과수정의템포가빨라서, 멋진전략보다자주끊어저장하는습관이더중요하다.
말하자면"정교한체스"보다"자주세이브"쪽에가깝다.

```
# 이 프로젝트의 커밋 패턴
git add -p     # 변경 내용을 확인하면서 스테이징
git commit -m "update"
```

커밋메시지가단순한 ("update") 것은의도적인선택이었다.

3 일간의빠른반복에서메시지작성에시간을쓰지않았다.
실제프로젝트라면의미있는메시지가좋겠지만, 프로토타이핑단계에서는 **커밋자체**가중요하다.

```
5404a78 update   ← GitHub Actions CI 추가
c4c439a update   ← Verilator 테스트벤치 추가
dfff9c6 update   ← RTL 버그 수정 (CRC, AXI)
dfed89d update   ← SRAM 모델, 합성 스크립트, 아키텍처 문서
2947352 update   ← 테스트 파라미터 추가
c1611de update   ← NCVerilog 테스트벤치
1072883 update   ← 호스트 소프트웨어
ca20700 initial  ← 사양서 + 전체 RTL (첫 커밋)
```

첫커밋 (`ca20700`) 에이미사양서와전체 RTL 이포함된것에주목하라.
사양서 → RTL 전환이얼마나빠르게이루어졌는지보여준다.

룰설정이 AI 출력품질을높이는방법

`project-instructions.md` 에넣은룰중실제로차이를만든것들:

용어표기룰

```
Master → Manager, Slave → Subordinate
불가피한 경우: M a s t e r, S l a v e (글자 사이 공백)
```

이룰덕분에생성된모든문서와주석이일관된용어를사용했다.
AI 에게일일이교정을요청하지않아도됐다.

영어작성룰

모든 작성물은 영어로 작성.

레지스터설명, 모듈주석, 사양서가모두영어로나왔다.

일관성유지룰

어느 하나에 수정이 있을 경우 프로젝트 내 모든 코드, 문서가 일관성을 유지하도록 함께 반영할 것.

레지스터맵이변경되면사양서, 드라이버헤더, 테스트벤치 task 를함께 수정하도록한것이다.
이룰이없으면코드가빠르게발산한다.

구조를먼저만드는것의가치

바이브코딩초보자가가장자주하는실수는 **구조없이바로코드를요청하는것**이다.

```
# 잘못된 시작
"AES decrypt IP Verilog 코드 만들어줘"
```

이렇게하면 AI 는어딘가에파일을만들지만, 프로젝트에통합되지않는다.
나중에테스트벤치를요청하면파일경로가맞지않고, 드라이버코드를요청하면레지스터주소가어긋난다.

```
# 올바른 시작
```

1. 폴더 구조를 만든다
2. project-instructions.md에 룰을 정의한다
3. 그 구조 안에서 파일을 하나씩 생성한다

구조가있으면 AI 의모든출력이 **그구조의일부**로생성된다.
그래서나중에테스트벤치, 드라이버, 문서가서로다른세계관에서태어나는일을줄일수있다.
이것이대규모 IP 를바이브코딩할때유지보수가능한코드베이스가만들어지는이유다.

다음장: 5 장—RTL 을바이브코딩하다

5 장—RTL 을바이브코딩하다

하루만에 9 개모듈이나왔다

2026 년 4 월 12 일, initial 커밋에포함된 RTL 파일목록:

```
design/rtl/
├── inc/aes_decrypt_defs.vh
├── crypto/aes128_key_sched.v
├── crypto/aes128_enc_pipe.v
├── crypto/aes128_ctr_top.v
├── util/sync_fifo.v
```

```
├── util/crc32_engine.v
├── aes_decrypt_regfile.v
├── aes_decrypt_desc_fetch.v
├── aes_decrypt_input_ctrl.v
├── aes_decrypt_output_ctrl.v
├── aes_decrypt_writeback.v
├── aes_decrypt_axi_mgr.v
├── aes_decrypt_ctrl.v
└── aes_decrypt_engine.v
```

14 개파일. 전체 RTL 코드베이스가한번의작업세션에서만들어졌다.
이문장을처음보면약간허풍처럼들릴수있다. 그런데실제로해보면, "완성"의의미를엄격하게잡더라도놀랄만큼빠르다.
물론그뒤에는리뷰와수정이기다리고있다. 버그는늘늦지않게도착한다.

아키텍처결정: 어떻게분해할것인가

전체를한번에요청하지않았다. 먼저아키텍처를 AI 와논의했다.

사용자 →

spec/AES-Decrypt-IP-Specification.md를 읽었지?
RTL 구현을 시작하기 전에, 어떤 모듈들로 나눌지 제안해줘.
AXI Manager 인터페이스를 어떻게 구현할지도 포함해서.

Claude →

사양서를 바탕으로 다음 모듈 구조를 제안합니다:

최상위 계층:

- aes_decrypt_engine.v : 배선 전용 최상위

제어 계층:

- aes_decrypt_ctrl.v : STOP/ACTIVE/PAUSE FSM + 파이프라인 오케
- aes_decrypt_regfile.v : AXI4-Lite 레지스터 파일

데이터 경로:

- aes_decrypt_desc_fetch.v : Descriptor 읽기 (AXI read, ring buf
- aes_decrypt_input_ctrl.v : 입력 버퍼 읽기 (AES Header + ciphert
- aes_decrypt_output_ctrl.v : 출력 버퍼 쓰기 (plaintext + padding
- aes_decrypt_writeback.v : Descriptor 상태 쓰기 (byte-strobe write)
- aes_decrypt_axi_mgr.v : AXI4 Manager (2개 Read 포트 + 2개 Wri

암호화 코어:

- aes128_key_sched.v : 키 스케줄 (순수 조합 논리)
- aes128_enc_pipe.v : 10라운드 파이프라인 (10클록 지연)
- aes128_ctr_top.v : CTR 모드 (카운터 블록 생성 + XOR)

유틸리티:

- crc32_engine.v : CRC-32 (IEEE 802.3 / CRC-32C 선택 가능)

- sync_fifo.v : 범용 동기 FIFO (SRAM 인터페이스)
- sram_2p_32x64.v : SRAM 행동 모델 (cipher FIFO용)
- sram_2p_32x72.v : SRAM 행동 모델 (output FIFO용)

AXI Manager는 4개의 독립 포트를 단일 AXI4 인터페이스로 중재합니다
우선순위: RD0(Descriptor) > RD1(Input), WR0(Writeback) > WR1(Outp

이구조제안을검토하고, 사양서와대조해서문제가없다고판단한후 RTL 작성을시작했다.
중요한점은 AI 가제안했고사람이승인했다는흐름이다. "AI 가알아서 했다"보다"AI 가여러초안을빠르게내고사람이고른다"가실제에가깝다.

[IMG] **[그림 5-1]** Claude Code 가아키텍처모듈구조를제안하는대화화면
텍스트응답아래에"Create these files?"라는확인프롬프트가표시된다

핵심모듈살펴보기

1. aes_decrypt_defs.vh —프로젝트의공통언어

모든모듈이공유하는파라미터와상수를여기에정의했다.

```
// Descriptor state codes — spec Table 6.3
`define DSTATE_NOT_PROCESSED  8'h00
`define DSTATE_OK             8'h01
```

```
`define DSTATE_CRC_ERR        8'h02
`define DSTATE_RD_ERR         8'h03
`define DSTATE_WR_ERR         8'h04
`define DSTATE_IN_PROGRESS    8'hFF

// Engine top-level states — spec Table 7.1
`define ENG_STOP    2'b00
`define ENG_ACTIVE  2'b01
`define ENG_PAUSE   2'b10
```

이파일이있으면다른파일들이매직넘버를직접쓰지않아도된다.
사양서의섹션번호까지주석으로달아서추적가능하게했다.
나중에코드를읽을때"이숫자가왜여기있지?"대신"아, spec 의그표에서왔구나"라는식으로맥락이이어진다.

---

2. AES 파이프라인코어

AES-128 CTR 모드의핵심은 10 라운드암호화파이프라인이다.

```
// aes128_enc_pipe.v —핵심 구조
module aes128_enc_pipe (
    input  wire          clk,
    input  wire          rst_n,
    input  wire          in_valid,
    input  wire [127:0]  in_block,     // 카운터 블록
```

```
    input  wire [1407:0] round_keys,   // 키 스케줄 (11 x 128-bi
    output wire          out_valid,
    output wire [127:0]  out_block     // 암호화된 블록 (= keyst
);
    // 10-stage pipeline: 각 스테이지가 AES 1 라운드를 처리
    reg [127:0] stage [0:9];
    reg         valid_pipe [0:9];

    // Stage 0: AddRoundKey (Initial)
    // Stage 1-9: SubBytes + ShiftRows + MixColumns + AddRoundK
    // Stage 10 (out): SubBytes + ShiftRows + AddRoundKey (Fina
```

10 클록지연의완전파이프라인구조다. 매클록마다새로운블록을입력받을수있다.
겉으로는조용한모듈이지만, 안에서는상당히부지런하다.
200 Mbps 목표를달성하려면이파이프라인이끊임없이채워져있어야한다. 결국성능은"코어가얼마나빠르냐"보다"굶기지않느냐"의문제로바뀐다.

CTR 모드에서는 AES **Encrypt** 함수를복호화에도동일하게사용한다.

`Decrypt: P_i = C_i XOR AES_Encrypt(Key, Counter_Block_i)`

`aes128_ctr_top.v` 가이를조율한다.

```
// aes128_ctr_top.v — CTR 모드 핵심 로직 (간략화)
// Counter block = { Nonce[95:0], (InitialCounter + i) mod 2^32
```

```
always @(posedge clk) begin
    if (aes_job_start) begin
        ctr_value <= aes_initial_ctr;
    end else if (enc_in_valid && enc_in_ready) begin
        ctr_value <= ctr_value + 1'b1;  // 32-bit wrap
    end
end

// 10 클록 지연된 ciphertext 와 keystream 을 XOR
assign plaintext = cipher_delay[9] XOR out_block;
```

---

3. 최상위 FSM —aes_decrypt_ctrl.v

이모듈이전체파이프라인을지휘한다.

사양서의 7.3 절"Detailed State Transition Rules"가코드로변환된것이다.

```
// 내부 FSM 상태 (TOP 레벨)
localparam TOP_STOP        = 4'd0;
localparam TOP_FETCH       = 4'd1;   // Descriptor 읽기 중
localparam TOP_INTERVAL    = 4'd2;   // valid=0, interval 대기
localparam TOP_WB_INPROG   = 4'd3;   // in-progress 상태 쓰기
localparam TOP_JOB_RUN     = 4'd4;   // 파이프라인 실행 중
localparam TOP_CRC_CHECK   = 4'd5;   // CRC 비교
```

```
localparam TOP_WAIT_OUT     = 4'd6;   // output_ctrl 완료 대기
localparam TOP_WB_FINAL     = 4'd7;   // 최종 결과 쓰기
localparam TOP_CHECK_FLAGS = 4'd8;   // interrupt/last 플래그 처
localparam TOP_PAUSE        = 4'd9;   // PAUSE 상태
localparam TOP_BUS_ERR      = 4'd10;  // 버스 에러 처리
localparam TOP_IMM_STOP     = 4'd11;  // 즉시 정지
```

사양서의 STOP/ACTIVE/PAUSE 3-상태모델이내부적으로는 12 개의마이크로상태로구현된것이다.

문서에는단순하게보이지만, 실제 RTL 은늘조금더수다스럽다. 그래야모듈간타이밍과책임이분명해진다.

이는표준적인하드웨어설계패턴이다. 외부에보이는상태와내부구현상태를분리하는편이결국디버깅에도유리하다.

> [IMG] **[그림 5-2]** aes_decrypt_ctrl.v 의 FSM 전환코드가에디터에표시된화면
> Claude Code 가코드를생성한직후, 변경사항이 diff 뷰로 표시된다

---

4. AXI Manager —가장복잡한모듈

`aes_decrypt_axi_mgr.v` 는 4 개의독립적인요청자 (Descriptor fetch, Input read, Writeback, Output write) 를하나의 AXI4 인터페이스로중재한다.

사용자 →

aes_decrypt_axi_mgr.v를 작성해줘.
2개의 Read 포트(RD0: desc_fetch, RD1: input_ctrl)와
2개의 Write 포트(WR0: writeback, WR1: output_ctrl)를 중재해야 해.

우선순위:
- Read: RD0 > RD1 (fixed priority)
- Write: WR0 > WR1 (fixed priority)

Outstanding 트랜잭션 수는 레지스터 설정값으로 제한해야 해.
사양서 12.3, 12.4절 참고.

모듈인터페이스만봐도복잡도를가늠할수있다.

```
module aes_decrypt_axi_mgr (
    input  wire        clk,
    input  wire        rst_n,

    // --- Outstanding 제한 설정 ---
    input  wire [4:0]  max_rd_outstanding,
    input  wire [4:0]  max_wr_outstanding,

    // --- Read Port 0 (Descriptor fetch & Writeback read) ---
    input  wire [31:0] rd0_addr,
    input  wire [7:0]  rd0_len,
    input  wire        rd0_req,
    output wire        rd0_grant,
```

```
    output wire [63:0] rd0_data,
    output wire        rd0_valid,
    output wire        rd0_last,
    output wire        rd0_err,

    // --- Read Port 1 (Input buffer read) ---
    input  wire [31:0] rd1_addr,
    input  wire [7:0]  rd1_len,
    input  wire        rd1_req,
    output wire        rd1_grant,
    // ... (동일 구조)

    // --- Write Port 0 (Descriptor writeback) ---
    // --- Write Port 1 (Output buffer write) ---
    // ...

    // --- AXI4 Manager 인터페이스 (단일 포트) ---
    output wire [31:0] m_axi_araddr,
    output wire        m_axi_arvalid,
    input  wire        m_axi_arready,
    // ... (전체 AXI4 신호)
);
```

Outstanding 트랜잭션카운터를구현하는부분:

```
// Read outstanding counter
reg [4:0] rd_outstanding_cnt;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        rd_outstanding_cnt <= 5'd0;
    end else begin
        // AR 채널에서 트랜잭션 발행 시 +1, RLAST 수신 시 -1
        case ({ar_accepted, rlast_received})
            2'b10: rd_outstanding_cnt <= rd_outstanding_cnt + 1
            2'b01: rd_outstanding_cnt <= rd_outstanding_cnt - 1
            default: ; // 동시 발생 또는 변화 없음
        endcase
    end
end

// Outstanding 제한: 카운터가 한계에 도달하면 arvalid 억제
assign can_issue_rd = (rd_outstanding_cnt < max_rd_outstanding)
```

---

5. 데이터흐름—파이프라인이어떻게동작하는가

한 Descriptor 의처리흐름을시간순으로보면:

T=0 desc_fetch가 AXI read 요청 → 32바이트 Descriptor 읽기

T=4 Descriptor 파싱 완료, in-progress 상태 쓰기(writeback) 시작
T=5 input_ctrl 시작: AES Header(16B) 먼저 읽기 → nonce, initial_
T=6 aes128_ctr_top에 job_start → 카운터 블록 생성 시작
T=7 ciphertext 읽기 시작 → cipher FIFO에 적재
T=7 CRC 계산 시작 (AES와 병렬)
T=8 cipher FIFO에서 데이터 → AES 파이프라인 입력
T=18 AES 파이프라인 첫 번째 출력 (10클록 지연)
T=18 output_ctrl 시작 → plaintext를 출력 버퍼에 쓰기
T=N 마지막 ciphertext 처리 → CRC 비교
T=N+1 writeback: 결과 코드(OK/CRC_ERR/...) 쓰기
T=N+2 다음 Descriptor로 이동

AES 와 CRC 가병렬로실행되는것 (T=7 에서동시시작), AES 파이프라인의 10 클록지연 (T=8 → T=18) 이데이터흐름의핵심이다.

AI 에게 RTL 을요청할때의프롬프트패턴

단순히"만들어줘"가아니라, 검증가능한제약을함께준다.
프롬프트가길어진다고해서나쁜것이아니다. 하드웨어에서는대체로그반대다.
말을아낄수록나중에파형을오래보게된다.

aes_decrypt_input_ctrl.v를 작성해줘.

담당 역할: 입력 버퍼를 AXI4 read로 읽어서

1. AES Header (첫 16바이트) → nonce, initial_ctr 추출
2. Ciphertext (IN_DATA_SIZE 바이트) → cipher FIFO에 전달
3. Padding (IN_PAD_SIZE 바이트) → 버림
4. CRC-32 값 (4바이트) → 래치 후 ctrl에 전달

요구사항:
- AXI 버스트는 최대 256비트 (AXI4 최대)
- AXI4 read 요청은 aes_decrypt_axi_mgr의 rd1 포트를 통해
- cipher FIFO가 가득 차면 더 이상 read 요청 보내지 말 것 (back-pressure)
- spec 11.2절 "No unnecessary bus congestion" 준수

인터페이스는 aes_decrypt_ctrl.v와 맞춰줘.

구체적인제약을함께주면 AI 가사양을놓치지않는다.

첫커밋의코드통계

```
$ find design/rtl -name "*.v" -o -name "*.vh" | \
  xargs wc -l | sort -rn | head -5

    387  aes_decrypt_axi_mgr.v
    318  aes_decrypt_ctrl.v
    285  aes_decrypt_input_ctrl.v
```

```
    243  aes128_enc_pipe.v
    198  aes_decrypt_output_ctrl.v
```

가장복잡한모듈은예상대로 AXI Manager 와최상위 FSM 이다.
총 RTL 코드라인수는약 2,800 라인. 한작업세션의결과물이다.

코드리뷰: AI 가스스로챙긴것들

생성된코드를검토하면서 AI 가자율적으로챙긴것들이눈에띄었다.

SVA Assertion 추가

`project-instructions.md` 에넣은룰대로, `ifdef` 조건부컴파일로 assertion 이포함되어있었다.

```
`ifdef ENABLE_ASSERTIONS
// AXI spec: ARVALID 이 한번 asserted 되면 ARREADY 전까지 내려가면
assert property (@(posedge clk) disable iff (!rst_n)
    (m_axi_arvalid && !m_axi_arready) |=> m_axi_arvalid)
else $error("AXI AR channel: arvalid dropped before arready");
`endif
```

헤더주석형식일관성

모든 14 개파일에동일한형식의헤더가있다.

```
// ============================================================
// File        : aes_decrypt_ctrl.v
```

```
// Project     : AES Decryption Engine IP
// Company     : SSVD
// Description : Top-level control FSM ...
// ==========================================================
```

리셋초기값주석

레지스터선언부에리셋값을명시하는주석이달려있다.

```
reg [1:0]  status_state;     // reset: ENG_STOP (2'b00)
reg [9:0]  cmd_head_ptr;     // reset: 10'd0
```

한계: AI 가놓친것

물론완벽하지않았다. 초기버전에서발견된문제들:

1. **sync_fifo 의 `wr_ptr` 이중구동** —두개의 `always` 블록이같은레지스터를구동
2. **CRC 엔진의초기화타이밍** —새 Descriptor 시작시초기화가 1 클록늦었음
3. **AXI Manager 의 Outstanding 카운터경계조건** —동시발행 + 완료시카운터가틀림

이버그들은다음날발견하고수정했다. 8 장에서자세히다룬다.

중요한것은, **이정도버그는사람이처음 RTL 을짜도나오는수준**이라는 것이다.

AI 가만든코드라서특별히버그가많은것이아니다.
오히려구조적인실수 (AXI 핸드셰이크위반, 래치생성등) 는없었다.
완벽하진않았지만, 적어도"처음부터다시써야한다"는종류의재앙은아니었다.

다음장: 6 장—레퍼런스소프트웨어를만들다

6 장—레퍼런스소프트웨어를만들다

왜소프트웨어가먼저인가

하드웨어 IP 를검증하려면 **정답데이터**가필요하다.
테스트벤치는"IP 가올바른출력을내보냈는가"를확인해야하는데,
"올바른출력"이무엇인지알려면소프트웨어레퍼런스구현이있어야한다.

즉, RTL 을믿기전에먼저비교할기준을만들어야한다.
검증의세계에서는"느낌상맞는것같다"가가장비싼문장이다.

RTL 과소프트웨어가동일한입력에대해동일한출력을내면, RTL 이맞다.
이원칙이 `host_software/`의존재이유다.

```
host_software/
├── aes128_ctr.c/.h    ← Pure C AES-128 CTR 구현 (레퍼런스)
├── crc32.c/.h         ← Pure C CRC-32 구현 (레퍼런스)
```

```
├── aes_decrypt_ip.c/.h ← IP 레지스터 드라이버
├── test_vectors.h      ← 테스트 입력/출력 벡터
├── tc_params.h         ← 테스트 케이스 파라미터
├── sw_test.c           ← 레퍼런스 SW만으로 검증
└── ip_test.c           ← IP 드라이버를 사용한 검증
```

Pure C 레퍼런스구현

사용자 →

host_software/aes128_ctr.c와 aes128_ctr.h를 작성해줘.
AES-128 CTR 모드 순수 소프트웨어 구현이야.

요구사항:
- 특정 CPU 가정 없이 generic하게 작성 (embedded 환경 고려)
- NIST SP 800-38A 표준 준수
- 헤더 포맷: Nonce(96-bit) + Initial Counter(32-bit)
- 카운터는 big-endian, 32-bit wrap
- hw 구현과 byte-for-byte 동일한 결과를 내야 함

생성된코드의핵심부분은비교적교과서적이다.
이런종류의코드는화려함보다해석가능성이중요하다. "잘돌아간다"보다"왜이렇게돌았는지읽힌다"가더값지다.

```
/* aes128_ctr.c — CTR 모드 블록 처리 */
void aes128_ctr_crypt(
```

```
    const uint8_t key[16],
    const uint8_t nonce[12],   /* 96-bit nonce */
    uint32_t      initial_ctr, /* 32-bit initial counter */
    const uint8_t *in,
    uint8_t       *out,
    size_t         len)
{
    uint8_t  counter_block[16];
    uint8_t  keystream[16];
    uint32_t ctr = initial_ctr;

    /* Counter block 구성: nonce(12B) || counter(4B, big-endian
    memcpy(counter_block, nonce, 12);

    while (len > 0) {
        /* Big-endian counter 삽입 */
        counter_block[12] = (uint8_t)(ctr >> 24);
        counter_block[13] = (uint8_t)(ctr >> 16);
        counter_block[14] = (uint8_t)(ctr >>  8);
        counter_block[15] = (uint8_t)(ctr      );

        /* AES Encrypt(counter_block) → keystream */
        aes128_encrypt_block(key, counter_block, keystream);

        /* XOR with input */
```

```
        size_t block_len = (len < 16) ? len : 16;
        for (size_t i = 0; i < block_len; i++)
            out[i] = in[i] ^ keystream[i];

        in  += block_len;
        out += block_len;
        len -= block_len;
        ctr++;   /* 32-bit wrap 은 자동 */
    }
}
```

IP 드라이버구조

IP 드라이버는 **플랫폼독립적 (generic)** 으로설계했다.
레지스터접근함수를함수포인터로주입받는방식이다.

```
/* aes_decrypt_ip.h —드라이버 컨텍스트 */
typedef struct {
    uint32_t  base_addr;   /* IP 레지스터 베이스 주소 */
    uint32_t  ring_phys;   /* Descriptor 링 버퍼 물리 주소 */
    uint32_t  ring_size;   /* 링 버퍼 슬롯 수 */
    uint32_t  tail;        /* 현재 tail 포인터 */

    /* 플랫폼 별로 제공하는 레지스터 접근 함수 */
```

```
    void     (*reg_write)(uint32_t base, uint32_t offset, uint3
    uint32_t (*reg_read) (uint32_t base, uint32_t offset);
} aes_ip_ctx_t;
```

이구조덕분에동일한드라이버코드가: - 시뮬레이션환경에서는테스트벤치레지스터접근함수를사용하고, - 실제 SoC 에서는 MMIO 접근함수를사용할수있다.

한마디로, 드라이버가특정보드나특정런타임에묶이지않는다.
초기프로젝트에서이런추상화를해두면나중에"이거왜테스트벤치에서만되죠?"같은질문을줄일수있다.

초기화함수:

```
int aes_ip_init(aes_ip_ctx_t *ctx,
                uint32_t max_rd_out, uint32_t max_wr_out)
{
    /* 엔진이 STOP 상태인지 확인 */
    uint32_t st = rreg(ctx, AES_IP_REG_STATUS)
                  & AES_IP_STATUS_STATE_MASK;
    if (st != AES_IP_STATE_STOP)
        return -1;

    /* 링 버퍼 설정 */
    wreg(ctx, AES_IP_REG_CMD_BUF_ADDR, ctx->ring_phys);
    wreg(ctx, AES_IP_REG_CMD_BUF_SIZE, ctx->ring_size);
    wreg(ctx, AES_IP_REG_CMD_TAIL_PTR, 0u);
```

```
    ctx->tail = 0u;

    /* AXI outstanding 제한 설정 (1-16 범위로 클램프) */
    uint32_t rd = (max_rd_out < 1u) ? 1u :
                  (max_rd_out > 16u) ? 16u : max_rd_out;
    uint32_t wr = (max_wr_out < 1u) ? 1u :
                  (max_wr_out > 16u) ? 16u : max_wr_out;
    wreg(ctx, AES_IP_REG_AXI_OUTSTAND,
         (rd << AES_IP_OUTSTAND_RD_SHIFT) |
         (wr << AES_IP_OUTSTAND_WR_SHIFT));

    return 0;
}
```

테스트벡터생성

테스트데이터는 Pure C 코드로직접생성했다. 키, nonce, plaintext 를 정하고, 암호화해서 ciphertext 를만들고, CRC 를계산한다.
손으로만들수도있겠지만, 그방식은보통두번째케이스쯤부터사람의집중력을시험하기시작한다.

사용자 →

test_vectors.h를 작성해줘.
AES-128 CTR 모드로 암호화된 테스트 데이터가 담겨야 해.

테스트 케이스:

1. 단순 케이스: 32바이트 plaintext (패딩 없음)
2. 패딩 케이스: 40바이트 plaintext (8바이트 패딩)
3. CRC 오류 케이스: 올바른 ciphertext지만 CRC 값이 틀림

모든 값은 C 배열로, 주석으로 바이트 의미를 설명해줘.

```
/* test_vectors.h —테스트 케이스 1 */
static const uint8_t TC1_KEY[16] = {
    0x2b, 0x7e, 0x15, 0x16, 0x28, 0xae, 0xd2, 0xa6,
    0xab, 0xf7, 0x15, 0x88, 0x09, 0xcf, 0x4f, 0x3c
};  /* AES-128 NIST test key */

static const uint8_t TC1_NONCE[12] = {
    0xf0, 0xf1, 0xf2, 0xf3, 0xf4, 0xf5, 0xf6, 0xf7,
    0xf8, 0xf9, 0xfa, 0xfb
};  /* 96-bit nonce */

static const uint32_t TC1_INITIAL_CTR = 0xfc000001u;

static const uint8_t TC1_PLAINTEXT[32] = { /* ... */ };
static const uint8_t TC1_CIPHERTEXT[32] = { /* ... */ };
static const uint32_t TC1_CRC32 = 0xA1B2C3D4u;  /* CRC over cip
```

드라이버와사양서사이의교차검증

드라이버를작성하면서사양서에모호한부분이발견되기도했다.

예를들어, AES 키를쓸때 byte order 가명확하지않았다.

발견된모호함:

```
사양서 5.9: "AES_KEY_0 holds key bits [31:0]"
그런데 key[0]이 LSB인가, MSB인가?
```

Claude 에게확인 →

```
aes_decrypt_ip.c의 aes_ip_write_key 함수에서
키 바이트 순서를 어떻게 해야 하는지 명확히 해줘.
RTL의 aes128_key_sched.v와 바이트 순서가 일치해야 해.
```

Claude 답변 →

```
RTL aes128_key_sched.v를 확인해보면:
   assign round_keys[31:0] = aes_key[31:0];  // = AES_KEY_0

즉 AES_KEY_0의 [31:0]이 키의 최하위 32비트입니다.
key[0]은 LSB, 즉 little-endian입니다.

드라이버에서:
   w = key[0] | (key[1]<<8) | (key[2]<<16) | (key[3]<<24);
    wreg(AES_KEY_0, w);
```

이런교차검증이드라이버작성과정에서자연스럽게이루어졌다.

흥미로운점은, 문서하나를쓰는일처럼보여도실제로는사양서와 RTL 과드라이버가서로를교정하고있다는것이다.
RTL 과드라이버가같은 AI 에의해만들어졌기때문에의도한바이트순서도일관되게유지됐고, 사람이그연결을확인하면서신뢰도를높일수있었다.

소프트웨어개발이 RTL 개발에미치는영향

Pure C 레퍼런스를먼저완성하고나니, 테스트데이터도준비됐고, 예상출력도생겼다.
이순간부터프로젝트는"설계를시작했다"에서"이제틀린걸잡아낼수있다"단계로넘어간다.
이것이다음단계인시뮬레이션환경구축의기반이된다.

`sw_test.c`가 검증하는 것: 소프트웨어 AES + CRC 구현의 정확성
`ip_test.c`가 검증하는 것: IP 드라이버 + IP 동작의 정확성

두 결과가 일치하면 IP가 올바르다.

다음장: 7 장—검증환경을구축하다

7 장—검증환경을구축하다

테스트벤치도바이브코딩한다

RTL 설계를마쳤다면이제시뮬레이션으로검증할차례다.
테스트벤치역시 AI 가만들었다. 그리고이역시하루만에구조를갖췄다.

여기서부터는프로젝트의분위기가조금달라진다.
코드를"만드는"단계에서, 코드를"의심하는"단계로넘어가기때문이다.
엔지니어에게는대체로이쪽이더익숙하다.

시뮬레이션환경의구조

```
┌──────────────────────────────────────────┐
│          tb_top.v (최상위)                 │
│                                          │
│  ┌──────────────┐   ┌──────────────────┐ │
│  │ Fake Host    │   │    DUT           │ │
│  │ CPU          │   │ (aes_decrypt_    │ │
│  │              │   │  engine.v)       │ │
│  │ AXI4-Lite    │◄──►                  │ │
│ │ Manager      │   │ AXI4 Manager ◄────┼┼──┐
│  │              │   │                  │ │  │
│  │ IRQ Handler  │◄──┤ irq              │ │  │
│ └──────────────┘   └──────────────────┘ │  │
│                                          │  │
│ ┌──────────────────────────────────────┐│  │
```

```
│  │        Fake Memory                  ||◄┘
│  │  (AXI4 Subordinate + readmemh)       ||
│  └──────────────────────────────────────┘|
└──────────────────────────────────────────┘
```

세구성요소: - **Fake Host CPU** —AXI4-Lite write/read task 로레지스터제어, 인터럽트처리 - **DUT** —`aes_decrypt_engine.v` (검증대상) - **Fake Memory** —AXI4 Subordinate, `readmemh` 로초기화, DUT 의 AXI4 Manager 요청처리

구조만보면단순해보이지만, 사실상작은 SoC 를축소모형으로만든셈이다.
좋은테스트벤치는 DUT 만똑똑한게아니라, 주변세계도그럴듯해야한다.

Fake Memory 설계—fake_mem.v

사용자 →

```
design/tb/fake_mem.v를 작성해줘.
AXI4 Subordinate로 동작하는 간단한 메모리 모델이야.

요구사항:
- 메모리 크기: 파라미터로 설정 (기본 64KB)
- readmemh로 초기화 가능
- AXI4 burst 지원 (INCR 타입)
```

- 64-bit 데이터폭 (DUT AXI Manager와 맞춤)
- 랜덤 지연 가능 (ready를 랜덤하게 de-assert, 실제 메모리 동작 시뮬
- 시뮬레이션 전용이므로 합성 가능성 불필요

핵심부분은메모리모델이너무똑똑하지도, 너무멍청하지도않게만드는데있다.
현실적인지연은주되, 분석불가능할정도로혼란스럽지는않아야한다.

```
// fake_mem.v — AXI4 Read 응답 처리 (간략화)
always @(posedge clk) begin
    if (ar_accepted) begin
        // 버스트 읽기 트랜잭션 등록
        rd_addr  <= axi_araddr;
        rd_len   <= axi_arlen;
        rd_count <= 8'd0;
        rd_busy  <= 1'b1;
    end

    if (rd_busy && r_accepted) begin
        // 다음 Beat 주소 계산 (INCR burst)
        rd_addr  <= rd_addr + 8;     // 64-bit = 8 bytes/beat
        rd_count <= rd_count + 1'b1;

        if (rd_count == rd_len) begin  // 마지막 beat
            rd_busy  <= 1'b0;
            axi_rlast <= 1'b1;
```

```
        end
    end
end

// 메모리 읽기 (64-bit aligned)
assign axi_rdata = mem_array[rd_addr[ADDR_WIDTH-1:3]];
```

메모리초기화파일생성—gen_mem.c

테스트데이터를메모리에배치하는것도자동화했다.
직접손으로집어넣기시작하면, 금방"이바이트를내가왜여기넣었지?"
라는질문과마주치게된다.

`gen_mem.c` 는 C 프로그램으로, 다음을수행한다.

1. `test_vectors.h` 에서테스트데이터읽기
2. 메모리레이아웃계산 (Descriptor 위치, 입력버퍼위치, 출력버퍼위치)
3. AES Header, ciphertext, padding, CRC 를입력버퍼에배치
4. `mem_init.hex` 파일출력 (`$readmemh` 형식)

```
/* gen_mem.c —메모리 레이아웃 생성 (핵심 부분) */
#define DESC_BASE     0x00001000   /* Descriptor 링 버퍼 시작 */
#define IN_BUF_BASE   0x00002000   /* 입력 버퍼 시작 */
#define OUT_BUF_BASE  0x00004000   /* 출력 버퍼 시작 */
```

```
/* Descriptor 0 작성 */
write32(mem, DESC_BASE + 0x00, 0x00000001);  /* valid=1 */
write32(mem, DESC_BASE + 0x04, IN_BUF_BASE);
write32(mem, DESC_BASE + 0x08, OUT_BUF_BASE);
write32(mem, DESC_BASE + 0x0C,
    (TC1_IN_PAD_SIZE << 24) | TC1_IN_DATA_SIZE);
write32(mem, DESC_BASE + 0x10,
    (TC1_OUT_PAD_SIZE << 24) | TC1_OUT_DATA_SIZE);

/* 입력 버퍼: AES Header 배치 */
memcpy(mem + IN_BUF_BASE + 0x00, TC1_NONCE, 12);
write32(mem, IN_BUF_BASE + 0x0C, TC1_INITIAL_CTR);

/* 입력 버퍼: Ciphertext 배치 */
memcpy(mem + IN_BUF_BASE + 0x10, TC1_CIPHERTEXT, TC1_IN_DATA_SI

/* 입력 버퍼: CRC 배치 */
write32(mem, IN_BUF_BASE + 0x10 + TC1_IN_DATA_SIZE, TC1_CRC32);
```

테스트시나리오—tb_core.sv

실제테스트시나리오는 tb_core.sv 에 task 로구현했다.
이파일은말하자면검증팀의대본이다. DUT 가어떤장면에서어떤반응

을보여야하는지한장면씩적혀있다.

사용자 →

design/tb/tb_core.sv를 작성해줘.
테스트 시나리오들이 담긴 파일이야.

다음 시나리오를 task로 구현해줘:
1. basic_decrypt_test: TC1 (정상 복호화)
2. crc_error_test: TC3 (CRC 오류 케이스)
3. interrupt_pause_resume_test: interrupt=1인 Descriptor 처리
4. last_descriptor_test: last=1인 Descriptor 후 STOP 확인
5. multi_descriptor_test: 여러 Descriptor 연속 처리

각 task는 레지스터 설정 → IP 시작 → 완료 대기 → 결과 검증 순서로.
검증 실패 시 $error로 메시지 출력.

```
// tb_core.sv —기본 복호화 테스트 (간략화)
task basic_decrypt_test;
    $display("[TEST] basic_decrypt_test start");

    // 1. AES 키 설정
    axil_write(AES_KEY_0, TC1_KEY_W0);
    axil_write(AES_KEY_1, TC1_KEY_W1);
    axil_write(AES_KEY_2, TC1_KEY_W2);
    axil_write(AES_KEY_3, TC1_KEY_W3);
```

```
// 2. 링 버퍼 설정
axil_write(CMD_BUF_ADDR, DESC_BASE);
axil_write(CMD_BUF_SIZE, 32'd4);    // 4-slot ring
axil_write(CMD_TAIL_PTR, 32'd1);    // 1 개 Descriptor 준비

// 3. IP 시작
axil_write(CTRL, 32'h1);            // CTRL.START=1

// 4. 완료 대기 (polling)
begin : wait_loop
    integer timeout;
    timeout = 10000;
    forever begin
        axil_read(STATUS, rd_data);
        if ((rd_data & 2'b11) == STATE_STOP) begin
            disable wait_loop;
        end
        @(posedge clk);
        if (--timeout == 0) begin
            $error("TIMEOUT: IP did not reach STOP");
            disable wait_loop;
        end
    end
end
```

```
    // 5. 결과 검증: 출력 버퍼 내용 확인
    for (int i = 0; i < TC1_OUT_DATA_SIZE; i += 8) begin
        expected = {TC1_PLAINTEXT[i+7], TC1_PLAINTEXT[i+6],
                    TC1_PLAINTEXT[i+5], TC1_PLAINTEXT[i+4],
                    TC1_PLAINTEXT[i+3], TC1_PLAINTEXT[i+2],
                    TC1_PLAINTEXT[i+1], TC1_PLAINTEXT[i+0]};
        actual   = fake_mem.mem_array[(OUT_BUF_BASE + i) >> 3]
        if (actual !== expected)
            $error("[FAIL] Output mismatch at +0x%0x: got %016h
                   i, actual, expected);
    end

    // 6. Descriptor 상태 확인
    desc_state = fake_mem.mem_array[DESC_BASE >> 3] >> 8;  // L
    if (desc_state !== 8'h01)
        $error("[FAIL] Descriptor state: got %02h, expected 0x0
               desc_state[7:0]);

    $display("[PASS] basic_decrypt_test");
endtask
```

NCVerilog 실행스크립트

```
# design/tb/run.sh
ncverilog \
  +access+r \
  +fsdb+all \
  -f rtl_filelist.f \
  -f tb_filelist.f \
  +incdir+../rtl/inc \
  +define+ENABLE_ASSERTIONS \
  tb_top.v \
  2>&1 | tee sim.log
```

`+define+ENABLE_ASSERTIONS` 를주면 RTL 에삽입된 SVA assertion 이활성화된다.
이어서션들이 AXI 프로토콜위반을시뮬레이션중에잡아낸다.
파형을두시간째보다가겨우알아낼실수를, 어서션은몇초만에"거기틀렸습니다"라고말해준다. 매너는없지만효율은좋다.

Verilator 테스트벤치—오픈소스로의전환

NCVerilog 는라이선스가필요한상용툴이다.
이후에오픈소스로 CI 를구축하기위해 Verilator 용테스트벤치도별도로만들었다.

Verilator 는 SystemVerilog → C++ 변환후컴파일해서실행하는방식이다.

C++ 드라이버가필요하다.

처음엔약간우회로처럼느껴지지만, CI 에올리기시작하면이선택이왜현실적인지곧체감하게된다.

```
// tb_dpi.cpp — Verilator 시뮬레이션 메인 루프 (핵심)
while (!context.gotFinish()) {
    // 클록 Rising Edge
    g_clk = 1;
    context.timeInc(kHalfPeriodNs);
    top->clk = g_clk;
    top->rst_n = g_rst_n;
    eval_step();

    // 클록 Falling Edge
    g_clk = 0;
    context.timeInc(kHalfPeriodNs);

    // 리셋 해제 타이밍
    if (context.time() >= kResetReleaseNs)
        g_rst_n = 1;

    top->clk = g_clk;
    eval_step();
```

```
}
```

SystemVerilog DPI 를통해 C++ 함수와 SystemVerilog task 가통신한다.

> [IMG] **[그림 7-1]** Verilator 빌드 + 시뮬레이션실행결과가터미널에출력되는화면
> *[PASS] basic_decrypt_test* 메시지가출력되고 exit code 0 으로종료

테스트벤치가잡아내는것

시뮬레이션환경이갖춰지면서검증할수있는항목들이눈에띄게늘어난다.
이시점부터는"돌아간다"보다"어디까지믿을수있는가"를말할수있게된다.

검증항목	방법
AES 복호화정확성	출력버퍼와레퍼런스 SW 결과비교
CRC 오류감지	잘못된 CRC 로설정한케이스에서 Descriptor 상태확인
인터럽트동작	PAUSE → IRQ → RESUME → ACTIVE 흐름시뮬레이션
last 플래그동작	마지막 Descriptor 후 STOP 확인
AXI 프로토콜	SVA assertion 이핸드셰이크위반감지
버스에러처리	fake_mem 에서 SLVERR 응답주입

다음장: 8 장—버그를찾고고치다

8 장—버그를찾고고치다

버그는반드시온다

어떤설계든첫번째버전에버그가없을수없다.
AI 가만든코드도마찬가지다. 중요한것은버그가있느냐없느냐가아니라,
버그를얼마나빠르게찾고, 얼마나정확하게고치는가다.

좋은소식은버그가나온다는사실자체가실패를뜻하지는않는다는점이다.
나쁜소식은, 그렇다고버그가스스로사라지지도않는다는점이다. 결국누군가는파형을봐야한다.

4 월 13 일저녁 ~14 일오전, 두번의커밋에서세가지버그가발견되고수정됐다.

```
dfed89d  update  ← SRAM 모델 추가, sync_fifo 수정, 합성 스크립트 추
dfff9c6  update  ← CRC 초기화 버그, AXI Manager 수정
```

각버그를순서대로살펴보자.

버그 1: sync_fifo 의이중구동 (Double Driver)

증상

합성툴을돌리자마자경고가나왔다.

```
Warning: Multiple drivers on net 'wr_ptr'
  1. Always block at sync_fifo.v:45
  2. Always block at sync_fifo.v:67
```

시뮬레이션에서는동작하는것처럼보였지만, 합성에서는구조적오류다.
이런종류의버그는특히얄밉다. 겉으로는멀쩡한데, 정작생산라인입구에서출입금지를당한다.

원인

초기 `sync_fifo.v` 에서 `wr_ptr` 을두개의 `always` 블록이동시에구동하고있었다.

```
// Before (버그 있는 코드)
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) wr_ptr <= 0;           // 리셋
end

always @(posedge clk) begin
    if (wr_en && !full) wr_ptr <= wr_ptr + 1;  // 쓰기
end
```

이것은 Verilog 에서동일한 reg 를두개의절차블록이구동하는고전적인실수다.
시뮬레이션에서는 rst_n=1 이면첫번째블록이아무것도하지않아우연히동작하지만, 합성에서는멀티플드라이버가되어버린다.

수정

```
// After (수정된 코드)
always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        wr_ptr <= 0;   // 리셋과 쓰기를 하나의 블록으로
    end else begin
        if (wr_en && !full)
            wr_ptr <= wr_ptr + 1;
    end
end
```

AI 와의수정과정

```
# 사용자 → Claude
sync_fifo.v에서 합성 툴이 wr_ptr 이중 구동 경고를 내고 있어.
두 개의 always 블록이 동일한 reg를 구동하는 문제야.
수정해줘.

# Claude 수정 후 메시지
wr_ptr을 단일 always 블록에서 관리하도록 수정했습니다.
async reset은 always @(posedge clk or negedge rst_n) 블록 안에서
```

if (!rst_n) 분기로 처리하는 것이 표준 패턴입니다.
rd_ptr, count도 동일한 패턴으로 되어 있는지 확인해보는 것이 좋겠습

[IMG] **[그림 8-1]** Claude Code 에서 sync_fifo.v 수정사항
이 diff 뷰로표시되는화면
삭제된라인은빨간색, 추가된라인은초록색으로표시되고,
Approve 버튼이있다

이과정에서 AI 가 rd_ptr, count 도같은패턴문제가있을수있다고미리알려준것에주목하자.
AI 는단순히요청한부분만고치는것이아니라, 같은패턴의다른문제도찾아낸다.
이럴때 AI 는"코드를써주는도구"라기보다"같이냄새를맡아주는리뷰어"에더가깝다.

버그 2: SRAM 포트인터페이스불일치

증상

테스트벤치를연결하려는데 FIFO 의 SRAM 포트가연결되지않았다.
초기 sync_fifo.v 에는외부 SRAM 포트가없었다—내부에 reg 배열을사용했다.

```
// Before (reg 배열 사용)
reg [DATA_WIDTH-1:0] mem [0:DEPTH-1];  // 내부 메모리
```

ASIC 환경에서는이것이문제다.

컴파일드 SRAM 을써야하는데, 내부 reg 배열은플립플롭으로합성된다.
MBIST 삽입도불가능하다.
시뮬레이션단계에서는편하지만, 실제칩관점에서는"지금은좋은데나중엔곤란한"전형적인선택이다.

해결방향

사용자 →

sync_fifo.v를 수정해서 외부 SRAM 인터페이스를 추가해줘.

ASIC 환경에서 컴파일드 SRAM을 쓰려면 FIFO 컨트롤러와 메모리를 분리
sync_fifo는 주소/데이터/enable 신호만 외부로 내보내고,
실제 메모리(sram_2p_32x64.v 등)는 aes_decrypt_engine.v에서 별도

이 구조로 변경하고, aes_decrypt_mem_top.v에 모든 SRAM을 모아서 MB

이수정이후구조가바뀌었다:

```
aes_decrypt_engine.v
├── aes_decrypt_input_ctrl.v
│   └── sync_fifo (cipher_fifo, 컨트롤러만)
│       ↕ SRAM 포트
└── aes_decrypt_mem_top.v  ← SRAM 집합체 (새로 추가)
    ├── sram_2p_32x64 (cipher FIFO storage)
    └── sram_2p_32x72 (output FIFO storage)
```

아키텍처문서 (doc/AES-Decrypt-IP-Architecture-Description.md) 에이변경이반영됐다:

```
## 9. Known Issues / Design Notes

2. SRAM read latency: The sram_2p_32x* behavioral models use as
   combinational read to match show-ahead FIFO behavior. If the
   provides only synchronous read, the sync_fifo controller mus
   to add a 1-cycle output pipeline register.
```

버그 3: CRC 엔진초기화타이밍

증상

CRC 오류테스트케이스를만들었는데, CRC 정상케이스에서도오류가 발생했다.
Descriptor 를여러개연속으로처리할때두번째 Descriptor 부터 CRC 값이틀렸다.

원인

CRC 엔진의 `crc_init` 신호가새 Descriptor 처리시작시 1 클록늦게 도착했다.

```
// Before: ctrl.v 에서 crc_init 타이밍
TOP_WB_INPROG: begin
```

```
    wb_start <= 1'b1;
    // ...
    crc_init <= 1'b1;  // ← 이 상태에서 crc_init
end

TOP_JOB_RUN: begin
    input_job_start <= 1'b1;
    crc_init <= 1'b0;  // ← JOB_RUN 진입 시 해제
end
```

문제는 `input_ctrl` 이 `JOB_RUN` 상태진입직후첫 ciphertext beat 를 CRC 엔진에보낼수있는데, `crc_init` 이이미 0 이되어이전 Descriptor 의 CRC 값이남아있을수있었다.

수정

```
// After: crc_init 을 JOB_RUN 시작 직전에 1 클록 asserted 상태로
TOP_WB_INPROG: begin
    wb_start  <= 1'b1;
    crc_init  <= 1'b1;  // CRC 초기화 시작
    // input_job_start 는 아직 내보내지 않음
end

TOP_JOB_RUN: begin
    // 이 상태 진입 시에도 crc_init=1 이 한 클록 보장됨
```

```
    input_job_start <= 1'b1;  // 이 사이클에 crc_init 은 아직 1
    crc_init        <= 1'b0;  // 다음 클록부터 0
end
```

이타이밍문제는시뮬레이션없이는발견하기어려운버그다.
테스트벤치에서"여러 Descriptor 연속처리"케이스를만들었기때문에 발견할수있었다.
단일케이스만돌렸다면꽤오래숨어있었을가능성이높다. 버그는늘가장 덜귀찮은테스트를좋아한다.

AI 와함께버그를디버깅하는방법

버그를발견했을때 AI 에게가장효과적으로전달하는방법:

방법 1: 증상 + 의심범위전달

```
CRC 테스트에서 두 번째 Descriptor의 CRC가 틀려.
첫 번째는 맞고, 두 번째부터 틀린다는 건 초기화 문제일 것 같아.
crc_init 신호의 타이밍을 aes_decrypt_ctrl.v에서 확인해줘.
```

방법 2: 시뮬레이션로그전달

```
시뮬레이션에서 이런 로그가 나왔어:

[TEST] multi_descriptor_test
[PASS] Descriptor 0: CRC OK
```

```
[FAIL] Descriptor 1: computed CRC=0xDEADBEEF, expected=0x1234567
```

CRC가 초기화되지 않고 누적되는 것 같아.
crc32_engine.v와 ctrl.v에서 crc_init 관련 코드를 보여줘.

방법 3: diff 를직접제시해서검토요청

이 diff를 검토해줘. 의도한 방향이 맞는지 확인해줘.

```
--- a/design/rtl/aes_decrypt_ctrl.v
+++ b/design/rtl/aes_decrypt_ctrl.v
@@ -142,7 +142,6 @@ TOP_WB_INPROG: begin
     wb_start  <= 1'b1;
     crc_init  <= 1'b1;
-    input_job_start <= 1'b1;  // 제거: 한 클록 늦춤
 end
 TOP_JOB_RUN: begin
+   input_job_start <= 1'b1;  // 여기서 시작: crc_init=1인 상태에
```

버그수정이가르쳐주는것

세가지버그의공통점: 1. **구조적오류** (이중구동) —문법/구조레벨 2. **아키텍처불일치** (SRAM 인터페이스) —설계의도레벨 3. **타이밍오류** (CRC 초기화) —동작레벨

AI 가생성한코드에서발생한버그의성격은사람이짜는코드와크게다르지않다.
차이가있다면, AI 는구조적오류보다타이밍/의도불일치버그를더자주낸다.
이것은 AI 가 **문법보다의미를처리하는데더어려움을겪기때문**이다.
겉보기엔그럴듯한데, 두세클록뒤에마음이틀어지는식의실수다.

그리고 AI 는버그를고치는것도빠르다.
문제를명확하게설명하면, 올바른수정안을수분안에제시한다.

---

commit dfed89d —이날추가된것들

버그수정외에도이커밋에서중요한것들이추가됐다.

```
design/syn/constraints.sdc   ← 200MHz 타이밍 제약 (SDC)
design/syn/run_dc.tcl        ← Synopsys DC 합성 스크립트
doc/AES-Decrypt-IP-Architecture-Description.md ← 아키텍처 문서
doc/compiled_memory_list.txt ← MBIST용 SRAM 목록
```

합성스크립트도 AI 가만들었다. 템플릿수준이지만구조는완전하다.

```
# design/syn/run_dc.tcl (핵심 부분)
# THROUGHPUT_TARGET: 200MHz 타이밍 제약
set CLK_PERIOD_NS 5.0   ;# 200 MHz = 5ns period
create_clock -period $CLK_PERIOD_NS [get_ports clk]

# 합성 타겟 (PDK 경로는 실제 환경에 맞게 수정 필요)
```

```
# set_attribute [get_lib *] default_threshold_voltage_group LV
```

다음장: 9 장—오픈소스 CI 로자동화하다

9 장—오픈소스 CI 로자동화하다

왜 CI 가필요한가

RTL 을손으로고쳐가면서매번시뮬레이션을수동으로돌리는방식에는문제가있다.
코드를수정했을때무언가부러졌는지즉시알수가없다.

작을때는괜찮다. "이번한번쯤은내가기억하지"라고생각할수있다.
하지만파일이늘고수정속도가빨라지면, 그기억력은생각보다금방파산한다.

소프트웨어세계에서는이것이 CI/CD 로해결된지오래다.
Push 할때마다자동으로테스트가돌고, 결과를알려준다.

하드웨어에도이패턴을적용할수있다.
상용 EDA 툴 (NCVerilog, VCS) 은 CI 환경에서쓰기어렵다. 라이선스서버가필요하고, 비용이크다.
그러나 **Verilator** 는오픈소스이고, GitHub Actions 에서무료로돌릴수있다.

물론상용시뮬레이터를완전히대체하는것은아니다. 하지만"코드가오늘도최소한의상식을지키는가"를확인하는데는꽤훌륭하다.

Verilator vs NCVerilog

항목	NCVerilog	Verilator
비용	상용 (고가)	무료오픈소스
CI 환경사용	어려움 (라이선스)	가능 (GitHub Actions)
지원언어	Verilog, SystemVerilog, VHDL	Verilog, SystemVerilog (일부 제한)
시뮬레이션속도	보통	매우빠름 (C++ 컴파일후실행)
파형포맷	VCD, FSDB	VCD, FST
SVA Assertion	지원	제한적

두환경이같은소스를공유한다는것이핵심이다.
NCVerilog 로개발·디버그하고, Verilator 로 CI 를돌린다.
로컬에서는편한도구를쓰고, 원격에서는반복가능한도구를쓰는식이다.
실용적이고, 무엇보다팀원설득이쉽다.

Verilator 테스트벤치구조

Verilator 는 Verilog 를 C++ 로변환해서컴파일한다.
따라서시뮬레이션드라이버를 C++ 로작성해야한다.
처음보면"왜갑자기 C++ 까지?"싶지만, 한번구조를잡아두면이후엔꽤 담백하게유지된다.

```
design/tb_verilator/
├── tb_top_verilator.sv  ← SystemVerilog 최상위 (DUT + Fake Memory
├── tb_dpi.cpp           ← C++ 시뮬레이션 드라이버
├── run.sh               ← Verilator 빌드 + 실행 스크립트
└── README.md
```

SystemVerilog DPI(Direct Programming Interface) 로 C++ 과 SystemVerilog 가통신한다.

```
// tb_top_verilator.sv
// DPI 함수로 클록과 리셋을 C++ 에서 제어
import "DPI-C" function bit tb_dpi_get_clk();
import "DPI-C" function bit tb_dpi_get_rst_n();

always @(*) clk   = tb_dpi_get_clk();
always @(*) rst_n = tb_dpi_get_rst_n();
```

C++ 드라이버에서클록을생성하고, 테스트완료신호를받는다.

```
// tb_dpi.cpp (핵심 루프)
while (!context.gotFinish()) {
```

```
    // Rising edge
    g_clk = 1;
    context.timeInc(kHalfPeriodNs);   // 5ns
    top->eval();
    trace->dump(context.time());

    // Falling edge
    g_clk = 0;
    context.timeInc(kHalfPeriodNs);   // 5ns

    // Reset release at t=80ns
    if (context.time() >= kResetReleaseNs)
        g_rst_n = 1;

    top->eval();
    trace->dump(context.time());
}

trace->close();
return context.gotFinish() ? 0 : 1;
```

---

run.sh —빌드와실행을한번에

```bash
#!/usr/bin/env bash
# design/tb_verilator/run.sh

set -euo pipefail
SCRIPT_DIR="$(cd "$(dirname "$0")" && pwd)"
RTL_DIR="${SCRIPT_DIR}/../rtl"
OUT_DIR="${SCRIPT_DIR}/out"
mkdir -p "$OUT_DIR"

# 1. Verilator: SystemVerilog → C++ 변환
verilator \
  --cc \
  --exe \
  --trace-fst \
  --timing \
  -Wno-WIDTH -Wno-CASEINCOMPLETE \
  -I"${RTL_DIR}/inc" \
  "${SCRIPT_DIR}/tb_top_verilator.sv" \
  "${SCRIPT_DIR}/tb_dpi.cpp" \
  --top-module tb_top_verilator \
  -CFLAGS "-O2" \
  -o "${OUT_DIR}/sim" \
  2>&1 | tee "${OUT_DIR}/verilator_build.log"
```

```
# 2. C++ 컴파일 + 링크
make -C obj_dir -f Vtb_top_verilator.mk 2>&1 | tee -a "${OUT_DI

# 3. 시뮬레이션 실행
"${OUT_DIR}/sim" 2>&1 | tee "${OUT_DIR}/verilator_console.log"
```

GitHub Actions 워크플로우

```yaml
# .github/workflows/verilator_ci.yml
name: Verilator Smoke Test

on:
  push:
    paths:
      - 'design/rtl/**'
      - 'design/tb_verilator/**'
      - 'host_software/**'

jobs:
  simulate:
    name: Verilator 5.043 / GCC 11 / Ubuntu 22.04
    runs-on: ubuntu-22.04
```

```yaml
    steps:
      - name: Checkout repository
        uses: actions/checkout@v4

      - name: Install system dependencies
        run: |
          sudo apt-get update -qq
          sudo apt-get install -y git autoconf flex bison gcc-1

      # Verilator 를 캐싱해서 매번 빌드하지 않도록
      - name: Cache Verilator 5.043 build
        id: cache-verilator
        uses: actions/cache@v4
        with:
          path: ~/verilator-install
          key: verilator-5.043-ubuntu-22.04-gcc11

      - name: Build Verilator 5.043
        if: steps.cache-verilator.outputs.cache-hit != 'true'
        run: |
          git clone --depth 1 --branch v5.043 \
            https://github.com/verilator/verilator /tmp/verilat
          cd /tmp/verilator
          autoconf && ./configure --prefix="$HOME/verilator-ins
          make -j$(nproc) && make install
```

```yaml
    - name: Run Verilator simulation
      run: |
        cd design/tb_verilator
        chmod +x run.sh && ./run.sh

    # 실패해도 아티팩트는 항상 저장
    - name: Upload simulation artifacts
      if: always()
      uses: actions/upload-artifact@v4
      with:
        name: sim-artifacts-${{ github.run_id }}
        path: |
          design/tb_verilator/out/verilator_console.log
          design/tb_verilator/out/dump.fst
        retention-days: 7
```

[IMG] **[그림 9-1]** GitHub Actions 에서워크플로우가성공적으로완료된화면
각단계 (Install, Cache, Build, Simulate) 에체크마크가표시되고총실행시간이보인다

워크플로우설계의핵심결정들

1. 트리거경로제한

```yaml
on:
  push:
    paths:
      - 'design/rtl/**'
      - 'design/tb_verilator/**'
```

RTL 이나테스트벤치가바뀔때만실행된다.
문서 (doc/, spec/, book/) 를수정할때는실행되지않는다.
불필요한 CI 실행을줄여 GitHub Actions 무료크레딧을절약한다.
CI 도공짜일때가장예민하게아껴쓰게되는법이다.

2. Verilator 빌드캐싱

Verilator 를소스에서빌드하면 10 분이상걸린다.
`actions/cache@v4` 로빌드결과물을캐싱하면두번째실행부터 30 초이내로줄어든다.
한번캐시를맛보면, 캐시없는 CI 로돌아가기가꽤어렵다. 인간은원래빠른것에쉽게적응한다.

```yaml
key: verilator-5.043-ubuntu-22.04-gcc11
```

버전을포함한캐시키를쓰면, Verilator 버전을바꿀때자동으로재빌드된다.

3. 아티팩트항상저장

```
if: always()
```

실패한경우에도로그와파형파일을저장한다.
CI 가실패했을때원인을분석할수있어야한다.

CI 구축프롬프트

사용자 →

```
.github/workflows/verilator_ci.yml을 작성해줘.

요구사항:
- Verilator 5.043 버전 고정
- Ubuntu 22.04에서 실행
- Verilator 빌드 결과를 캐싱해서 재빌드 방지
- design/rtl, design/tb_verilator 변경 시에만 트리거
- 시뮬레이션 실패해도 로그 파일은 artifact로 저장
- dump.fst 파형 파일도 저장 (7일 보관)
```

이프롬프트하나로완성된워크플로우파일이나왔다.
GitHub Actions 의문법, 캐싱전략, 아티팩트설정이모두포함된상태로.

CI 가가져다주는것

CI 를구축하고나서개발사이클이달라진다.
체감상가장큰변화는심리적안정감이다. "내가뭘망가뜨렸나?"를혼자짐작하는대신, 파이프라인이객관적으로대답해준다.

코드 수정 → `git push` → [자동] 빌드 + 시뮬레이션 → 결과 확인

> [IMG] **[그림 9-2]** VSCode Source Control 패널에서 commit + push 후 GitHub PR 체크상태표시
> "All checks passed"또는"1 check failed"배지가표시된다

이것이하드웨어개발에서갖는의미:
- 어떤모듈을수정해도, 기존테스트가여전히통과하는지자동으로확인된다 - 리그레션을두려워하지않고리팩토링할수있다 - 다른사람이코드를보내도검증을자동화할수있다

다음장: 10 장—다른 AI 를리뷰어로부르다

10 장—다른 AI 를리뷰어로부르다

같은 AI 에게만물으면생기는문제

Claude 와함께 3 일동안개발하면서한가지한계를체감했다.
정확히는, "혼자잘달리는 AI"에게도결국리뷰어는필요하다는사실을깨

닫게된다.

AI 도 **확증편향 (confirmation bias)** 이있다.
Claude 가만든코드를 Claude 에게리뷰요청하면, 자신이만든코드를 너무관대하게평가한다.
"이코드가올바른가?"라고물으면대부분"네, 맞습니다"라고답하거나, 사소한개선만제안한다.

진짜리뷰는 **독립적인시각**에서나온다.
사람팀에서도작성자와리뷰어를굳이나누는데는이유가있다. AI 라고 갑자기그원리를초월하진않는다.

해결책: **다른 AI 를리뷰어로투입한다.**
조금우스운그림처럼들릴수있다. AI 가짠코드를다른 AI 가보고, 그걸 사람이판정한다.
그런데막상해보면꽤합리적이다. 리뷰어가바뀌면질문의결도바뀌고, 놓친패턴도달라진다.

이프로젝트에서는 OpenAI **Codex** (또는 ChatGPT with code analysis) 를리뷰어로활용했다.
Claude 가만든코드를 Codex 에게보내서"이코드의문제점을찾아줘"라고요청했다.

Codex 리뷰를위한준비

리뷰요청시컨텍스트를충분히제공해야효과적이다.
좋은리뷰는좋은질문에서나오고, 좋은질문은대체로배경설명을아끼지

않을때나온다.

Codex 에게보낸리뷰요청구조:

다음은 AES-128 CTR 모드 복호화 하드웨어 IP의 Verilog RTL 코드야. ASIC 구현을 목표로 하고, AXI4 인터페이스를 사용해.

배경:
- Memory-to-memory 방식, Descriptor 기반 DMA
- AXI4 64-bit Manager (최대 outstanding 16개)
- AXI4-Lite Subordinate (레지스터 인터페이스)
- 목표 처리량: 200 Mbps at 200 MHz

다음 관점에서 코드 리뷰를 해줘:
1. AXI4 프로토콜 준수 여부 (핸드셰이크 타이밍, outstanding count
2. 합성 문제 가능성 (래치 생성, 조합 루프, 멀티 드라이버)
3. 리셋 동작 완전성 (모든 레지스터가 안전한 초기 상태)
4. 코너 케이스 (FIFO 경계 조건, 버스 에러 중 flush 동작)
5. 성능 병목 가능성

[코드 첨부]

Codex 가발견한주요이슈들

이슈 1: AXI AR 채널—arvalid 철회위험

Codex 코멘트:

aes_decrypt_axi_mgr.v에서 outstanding 카운터가 한계에 도달했을 때
arvalid를 즉시 de-assert합니다.

AXI4 스펙(섹션 A3.2.1):
"Once ARVALID is asserted, it must remain asserted until the risir
clock edge after ARREADY is asserted."

만약 arvalid가 asserted된 상태에서 (한계 초과 등으로) de-assert되면
AXI 스펙 위반입니다.

확인결과:
실제코드를보니, arvalid 가 grant 전에내려갈수있는경로가있었다.
이런버그는시뮬레이션에서바로터지지않을수도있어서더위험하다.
"가끔은된다"는종류의문제는늘오래산다.

Claude 에게수정요청:

aes_decrypt_axi_mgr.v에서 AXI 스펙 위반 가능성이 있어.
arvalid를 asserted한 후 arready 전에 de-assert하면 안 돼.
이걸 막는 로직을 추가해줘.

수정된코드:

```
// arvalid 는 한번 assert 되면 arready 전에 내려갈 수 없음
// pending_ar 플래그로 진행 중인 AR 트랜잭션 추적
reg pending_ar;

always @(posedge clk or negedge rst_n) begin
    if (!rst_n) begin
        pending_ar <= 1'b0;
    end else begin
        if (m_axi_arvalid && m_axi_arready)
            pending_ar <= 1'b0;   // 수락됨
        else if (m_axi_arvalid)
            pending_ar <= 1'b1;   // 발행 중, 철회 불가
    end
end

// arvalid: 새 요청이 있고 outstanding 한계 미달일 때, 또는 이미 pe
assign m_axi_arvalid = pending_ar ||
                       (rd_req_arb && can_issue_rd && !pending_
```

이슈 2: 버스에러중 Outstanding 트랜잭션처리

Codex 코멘트:

버스 에러 발생 시 "모든 outstanding 트랜잭션이 완료될 때까지 STOP

사양서에 명시되어 있습니다.

현재 코드(TOP_BUS_ERR 상태)에서는 rd_outstanding_cnt만 확인하고 wr_outstanding_cnt도 0이 될 때까지 기다려야 하지 않나요?

또한, SLVERR 응답이 읽기와 쓰기 동시에 발생하면 state가 어떻게 됩

확인결과:

쓰기 outstanding 카운터를드레인하지않고 STOP 으로진입하는경로가있었다.

단일에러만기준으로설계했고, 동시에러케이스를빠뜨렸다.

수정방향:

```
// TOP_BUS_ERR: 읽기와 쓰기 모두 드레인 완료 후 STOP
TOP_BUS_ERR: begin
    if (rd_outstanding_cnt == 5'd0 &&
        wr_outstanding_cnt == 5'd0) begin
        state <= TOP_STOP;
        status_state <= `ENG_STOP;
    end
    // 에러 발생 후 새 트랜잭션 발행 억제는 bus_err_flag 로 처리
end
```

이슈 3: CRC 엔진알고리즘전환타이밍

Codex 코멘트:

CRC_CTRL.ALG_SEL 레지스터가 사양서에 따르면
"Writes are accepted at any time; the selected algorithm applies to descriptors processed after the write."

그런데 현재 crc32_engine.v는 alg_sel 신호가 바뀌면 즉시 내부 상태
Descriptor 처리 중간에 ALG_SEL이 바뀌면 CRC 결과가 섞일 수 있습니

수정:

Descriptor 처리시작시 (TOP_WB_INPROG 상태) 에 alg_sel 을래치해서,
처리중간에레지스터가변경되더라도같은알고리즘으로끝까지처리하도록했다.
사양을글로읽으면당연해보이는데, 코드에서는의외로쉽게빠지는부분이다.

```
// Descriptor 처리 시작 시 alg_sel 래치
reg crc_alg_latched;

always @(posedge clk) begin
    if (state == TOP_WB_INPROG)
        crc_alg_latched <= crc_alg_sel;  // 시작 시점에 고정
end
```

```
// CRC 엔진에는 래치된 값을 사용
assign crc_alg = crc_alg_latched;
```

Codex 리뷰의가치—정량적으로

발견이슈	심각도	발견방법
AXI arvalid 철회위험	높음 (프로토콜위반)	Codex 리뷰
Write outstanding 드레인누락	중간 (버스에러시 undefined state)	Codex 리뷰
CRC 알고리즘중간전환	낮음 (엣지케이스)	Codex 리뷰

이중첫번째이슈 (AXI arvalid 철회) 는실제 SoC 에통합했을때다른 IP 와의연동에서간헐적버그를유발했을것이다.
재현하기어렵고원인을찾기도어려운종류의버그다. 흔히말하는"회의에서는재현안되고, 데모직전에만나오는"부류에가깝다.

AI 협업에서다중 AI 활용전략

이경험에서도출한원칙:

Claude (개발자역할) - 코드생성 - 리팩토링 - 설계결정지원

Codex (리뷰어역할) - 독립적인시각으로코드검토 - 프로토콜스펙준수확인 - 엣지케이스발굴

두 AI 의강점이다르고, 만들어내는오류의패턴도다르다.
한 AI 가놓친것을다른 AI 가잡아내는상호보완관계가된다.
사람팀에서성향다른리뷰어둘이붙으면품질이올라가는것과비슷한원리다.

리뷰어 AI 에게효과적으로요청하는법

무조건"이코드리뷰해줘"가아니다.

- "이 Verilog 코드 리뷰해줘"
  → 너무 막연함, 표면적인 것만 보게 됨

- "AXI4 프로토콜 스펙의 섹션 A3.2.1을 기준으로, arvalid/arready 핸드셰이크 처리에 위반이 있는지 확인해줘"
  → 구체적인 기준이 있어서 정밀한 리뷰가 가능

- "버스 에러 처리 경로에서 데드락 가능성이 있는지 추적해줘. 특히 outstanding 카운터가 0으로 내려오지 않는 케이스를 찾아줘"
  → 목표가 명확해서 AI가 집중할 수 있음

다음장: 에필로그—AI 동료와일한다는것

에필로그—AI 동료와일한다는것

---

3 일의기록을돌아보며

4월 12일: original-prompt.md → 사양서 → 전체 RTL (14개 파일)
4월 13일: 호스트 소프트웨어 → 시뮬레이션 테스트벤치 → 버그 수정 시
4월 14일: 추가 버그 수정 → Verilator 테스트벤치 → GitHub Actions C

사양서, RTL, C 드라이버, 테스트벤치, 합성스크립트, CI 파이프라인.
IP 개발의전체사이클이 3 일안에돌아갔다.

문장만보면약간과장처럼들릴수있지만, 실제체감은오히려더이상했다.
평소라면각각다른일정표에있던일들이한주안에서아니라, 하루단위로서로붙기시작했기때문이다.

혼자였다면몇주가걸렸을것이다. 팀이있었다면더빠를수도있었겠지만, 코드리뷰와인터페이스합의에서커뮤니케이션비용이발생한다.
AI 와의작업에는그비용이거의없다. 대신작성시간은줄고판단시간은늘어난다. 하드웨어엔지니어의무게중심이조금이동한다.

바이브코딩이잘맞는영역과그렇지않은영역

잘맞는영역:

- **표준프로토콜구현** —AXI, APB, AHB 같은스펙이명확한인터페이스. AI 가프로토콜을이미학습했기때문에세부구현이빠르다.
- **반복적인구조** —레지스터파일, FIFO, 상태머신. 패턴이명확해서 AI 가잘따라온다.
- **레퍼런스구현** —알고리즘의 SW 구현. AES, CRC 같은표준알고리즘은완성도가높다.
- **인프라코드** —빌드스크립트, CI 설정, 문서템플릿.

주의가필요한영역:

- **PDK 특화설계** —특정공정의타이밍특성, 셀선택. AI 는일반적인원칙만알고있다.
- **아날로그·혼합신호** —Verilog-A, 아날로그시뮬레이션. AI 가직접다루기어렵다.
- **최적화후반부** —PPA(Power-Performance-Area) 세밀한조정은결국사람의판단이필요하다.
- **도메인특수지식** —프로토콜이공개되지않은독점인터페이스, 내부팀만아는규약.

엔지니어의역할이바뀐다

바이브코딩이후, 하드웨어엔지니어의역할중심이이동한다.

Before:
"어떻게구현하는가"에시간의 70% 를쓴다.

After:

"무엇을만들것인가", "올바르게만들어졌는가"에시간을더쓴다.

이것은역할의격하가아니다. 더높은수준의판단이요구된다.
AI 가만든코드를검토하고, 방향이잘못됐을때수정하고, 엣지케이스를발굴하는능력, 이것이오히려더중요해진다.
예전에는손이먼저바빴다면, 이제는판단이먼저바빠진다.

도메인지식이없는사람은 AI 가만든코드가옳은지그른지판단할수없다.
바이브코딩은도메인지식을대체하지않는다. **도메인지식의레버리지를극대화**한다.
전문성이없는사람이갑자기칩을찍어내게되는이야기가아니라, 전문성이있는사람이훨씬넓은범위를다룰수있게되는이야기다.

이방식으로개발을시작하려는분께

첫번째단계: 작은모듈하나로시작하라.
처음부터전체 IP 를바이브코딩하려하지말고, FIFO 하나, 레지스터파일하나부터시작하라.
AI 가어떻게동작하는지체감한뒤범위를넓혀가는것이안전하다.

두번째단계: 컨텍스트파일을만들어라.
`project-instructions.md` 같은파일을만들어서확정된결정사항을기록하라.
AI 는이파일을기준으로일관된결과물을만든다.

세번째단계: 검증환경을일찍만들어라.

RTL 설계와테스트벤치를동시에진행하라. AI 는두작업을동시에할수있다.
검증이없으면 AI 의출력물이옳은지확인할방법이없다.

네번째단계: 다른 AI 의리뷰를받아라.
같은 AI 에게자신의코드를리뷰요청하는것은효과가제한적이다.
독립적인시각을가진다른 AI 를리뷰어로투입하면놓쳤던문제를발견할수있다.

마지막으로

이책은"AI 한테칩설계를시켜봤더니됐다"는이야기다.
동시에, "됐지만그냥되는게아니었다"는이야기이기도하다.
AI 는유능한신입이자지치지않는실무자처럼보일때가많지만, 여전히옆자리의시니어가필요하다.

방향을잡고, 구조를세우고, 결과를검토하고, 버그를분석하고, 다른 AI 를리뷰어로부르는과정—이모든것이엔지니어의몫이었다. AI 는빠르게실행했지만, 판단은사람이했다.

앞으로 AI 의능력은더빠르게발전할것이다. 이책에서다룬방식도 1~2 년안에더나은방법으로대체될것이다.
그래서더중요한것은특정툴의사용법보다협업의원리다. 어떤도구가오더라도, 결국남는것은질문하는방식과검증하는태도다.

중요한것은, **하드웨어엔지니어가이변화의주도권을갖는것**이다.
AI 를두려워하거나무시하는대신, 어떻게협업할것인지를배우는쪽이

훨씬유리하다.
도구는계속바뀌겠지만, 주도권은여전히문제를이해하는사람쪽에남는다.

이책이그첫걸음에도움이되었기를바란다.

부록: 이책에서사용한프롬프트패턴요약

상황	프롬프트패턴
새세션시작	"prompt/project-instructions.md 를 먼저 읽어줘"
새모듈생성	"[파일명] 을 작성해줘. 담당 역할: [...] 요구사항: [...]"
버그수정	" 증상 + 의심 범위 + 관련 파일명"
리뷰요청	"[구체적인 기준] 을 기준으로 [특정 코드] 를 리뷰해줘"
문서업데이트	"[파일 A] 변경에 맞게 [파일 B, C] 도 동기화해줘"
아키텍처논의	" 이 설계를 어떤 모듈로 나눌지 먼저 제안해줘"

「AI 한테칩설계시켜봤다」—끝